

Laporan Milestone 3

Semantic Analysis

IF2224 Teori Bahasa Formal dan Automata



Kelompok JEE - SundalnJVM

Billie Bhaskara Wibawa	13524024
Raysha Erviandika Putra	13524050
Muhammad Naufal Romi Annafi	13524058
Dzaki Ahmad Al Hussainy	13524084

PROGRAM STUDI TEKNIK INFORMATIKA

SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA

INSTITUT TEKNOLOGI BANDUNG

JL. GANESA 10, BANDUNG 40132

2026

Daftar Isi

Daftar Isi.....	2
1. Dasar Teori.....	3
1.1. Semantic Analysis.....	3
1.2. Abstract dan Decorated Syntax Tree.....	3
1.3. Attributed Grammar.....	4
1.4. Symbol Table dan Manajemen Scope.....	5
1.5. Visitor Pattern dalam Traversal Semantik.....	6
1.6. Type Checking dan Konsistensi Semantik.....	8
2. Perancangan dan Implementasi.....	10
2.1. Teknologi yang Digunakan.....	10
2.2. Alur Eksekusi Semantic Analyzer.....	10
2.3. Arsitektur Program.....	11
2.4. Kelas SymbolTable.....	13
2.5. Kelas ASTNode dan ASTBuilder.....	17
2.6. Perancangan Semantic Analyzer dengan Semantic Visitor.....	30
3. Pengujian.....	35
5. Kesimpulan dan Saran.....	54
6. Lampiran.....	56
7. Referensi.....	56

1. Dasar Teori

1.1. Semantic Analysis

Semantic Analysis merupakan tahapan dalam proses kompilasi yang berfokus pada pengecekan aturan logika program berdasarkan aturan bahasa yang berlaku. Jika syntax analysis bertugas memastikan kebenaran struktur atau bentuk tulisan, maka semantic analysis memastikan bahwa instruksi tersebut masuk akal secara fungsional. Proses ini dilakukan dengan mengolah struktur parse tree yang telah dihasilkan pada tahap sebelumnya.

Terdapat beberapa fungsi krusial dalam tahapan ini:

1. Validasi Identifier dan Deklarasi: Sistem menjamin bahwa setiap identitas seperti variabel, fungsi, atau konstanta sudah didefinisikan sebelum digunakan dan tidak ada deklarasi ganda yang tumpang tindih dalam satu lingkup yang sama.
2. Type Checking: Compiler memastikan konsistensi tipe data dalam setiap operasi. Sebagai contoh, operasi aritmetika hanya diizinkan pada tipe integer atau real, sedangkan pada tipe boolean dengan string akan ditolak.
3. Scope Management: Semantic analysis mengelola aturan lexical scoping untuk memastikan bahwa pemanggilan sebuah nama merujuk pada deklarasi yang tepat sesuai letak bloknya.
4. Decorated AST: AST yang semula polos akan diberikan anotasi atau informasi tambahan, seperti tipe hasil ekspresi dan referensi ke tabel simbol. Hasil akhir dari proses ini adalah decorated AST yang menjadi fondasi bagi tahap optimasi dan pembuatan kode mesin.

Secara keseluruhan, semantic analysis berperan sebagai penyaring kesalahan logika yang tidak tertangkap oleh parser, seperti ketidaksesuaian jumlah parameter fungsi, kesalahan akses indeks array, atau penggunaan variabel yang belum ada.

1.2. Abstract dan Decorated Syntax Tree

Abstract Syntax Tree (AST) merupakan representasi hierarkis dari struktur logika kode sumber yang telah disederhanakan dari parse tree. Jika parse tree mencatat setiap detail aturan tata bahasa sesuai dengan Context Free Grammar termasuk elemen-elemen seperti titik koma, tanda kurung, atau terminal yang hanya berfungsi sebagai pemisah, AST mengeliminasi bagian yang tidak penting tersebut dan fokus ke node-node penting dalam pengecekan logika bahasa.

Fokus utama AST adalah merepresentasikan hubungan fungsional antara operator dan operand secara ringkas. Dalam struktur ini, setiap simpul mewakili konstruksi program seperti pernyataan if, perulangan while, atau operasi aritmetika serta bentuk ekspresi atau tipe data. Hal ini dilakukan karena proses semantic analysis tidak membutuhkan node-node seperti titik koma sehingga pengecekan menjadi lebih efisien

Tahap Semantic Analysis mengambil Abstract Syntax Tree dan mentransformasikannya menjadi Decorated Abstract Syntax Tree atau melalui proses dekorasi. Proses ini melibatkan penyisipan informasi tambahan atau atribut pada setiap simpul pohon untuk memberikan konteks semantik yang lengkap.

Informasi yang ditambahkan selama proses dekorasi meliputi:

1. Pengecekan Tipe: Menentukan tipe data hasil dari setiap ekspresi, misalnya, menandai bahwa hasil dari $5 + 2.0$ adalah real.
2. Resolusi Identifier: Memastikan setiap variabel atau fungsi merujuk pada alamat atau entitas yang benar.
3. Pengelolaan Scope: Menandai tingkatan cakupan variabel apakah lokal atau global untuk memastikan akses data yang valid.

1.3. Attributed Grammar

Context-Free Grammar merupakan kerangka matematis yang menjadi landasan utama dalam mendefinisikan syntax suatu bahasa pemrograman. Sebuah CFG secara formal direpresentasikan melalui empat elemen utama atau 4-tuple $G = (N, T, P, S)$:

- Simbol Non-terminal (N): Kumpulan simbol yang berperan sebagai variabel dan dapat diuraikan lebih lanjut.
- Simbol Terminal (T): Elemen dasar bahasa yang tidak dapat diubah lagi, biasanya selaras dengan hasil identifikasi token oleh lexer.
- Aturan Produksi (P): Serangkaian instruksi yang mengatur bagaimana sebuah non-terminal dapat ditransformasikan menjadi deretan simbol terminal maupun non-terminal.
- Simbol Awal (S): Titik awal dari seluruh proses pembentukan tata bahasa.

Attributed grammar ada sebagai pengembangan dari CFG untuk menyisipkan semantic rule ke dalam struktur sintaksis. Dalam konsep ini, setiap simbol dalam tata bahasa diberikan atribut atau informasi tambahan yang krusial bagi proses kompilasi, seperti tipe data ekspresi, nilai konstanta, referensi pada tabel simbol, hingga scope variabel.

Berdasarkan cara pengalirannya, atribut ini dikategorikan menjadi dua jenis:

1. **Synthesized Attributes:** Nilai yang diperoleh dari hasil evaluasi simpul anak kemudian diteruskan naik ke simpul induk.
2. **Inherited Attributes:** Nilai yang didapat dari konteks di sekitarnya, yakni dari simpul induk atau simpul saudara, yang kemudian mengalir turun ke bawah atau menyamping.

Dalam praktik pembuatan compiler, model L-attributed grammar menjadi standar yang umum digunakan. Keunggulan utama model ini adalah kemampuannya untuk dievaluasi cukup dengan satu kali penelusuran dari kiri ke kanan. Dalam skema ini, inherited attribute diproses saat sistem bergerak menelusuri aturan produksi, sedangkan synthesized attribute diselesaikan setelah semua simpul anak dikunjungi.

1.4. Symbol Table dan Manajemen Scope

Dalam pembuatan compiler, Symbol Table adalah mekanisme untuk menyimpan dan mengelola semua informasi mengenai identifier yang muncul dalam kode sumber. Tabel simbol bertindak sebagai sumber data internal compiler yang menyimpan atribut penting seperti tipe, lokasi memori, dan cakupan akses.

A. tab

Tabel tab adalah sumber bagi semua identifier yang dideklarasikan oleh pengguna seperti variabel, konstanta, fungsi, serta predefined identifiers.

- **Identitas dan Klasifikasi:** Melalui atribut obj, compiler menentukan apakah sebuah nama adalah variabel, konstanta, atau prosedur. Atribut type menyimpan jenis datanya seperti integer atau real.
- **Scope:** Atribut lev menunjukkan kedalaman blok kode. Link adalah salah satu atribut yang menentukan scope dari tiap identifier, atribut ini bekerja seperti linked list yang menghubungkan satu identifier ke identifier sebelumnya dalam blok yang sama. Saat mencari sebuah variabel, compiler menelusuri rantai link ini dari scope terdalam hingga ke scope global.
- **Relasi Antar Tabel:** Jika sebuah identifier memiliki tipe kompleks, atribut ref akan menyimpan indeks yang mengarah

ke tabel pendukung misal atab untuk array atau btab untuk prosedur atau record.

B. atab

Tipe data array memiliki banyak detail teknis yang tidak muat disimpan di tabel utama sehingga digunakanlah atab atau Array Table.

- Struktur Indeks dan Elemen: Tabel ini mencatat batas bawah dan batas atas dari indeks array (misal: 1..10). Atribut xtyp menyimpan tipe data indeksinya, sementara etyp menyimpan tipe data elemen yang dikandungnya.
- Komposisi Kompleks: Jika kita memiliki array di dalam array atau array of records, atribut eref akan memberikan referensi lanjutan ke entri atab atau btab berikutnya.
- Manajemen Memori: Atribut elsz dan size sangat penting bagi tahap Code Generation untuk menghitung berapa banyak ruang memori yang harus dialokasikan di stack atau heap.

C. btab

Tabel btab atau block table digunakan untuk mengelola konteks setiap sub-program atau struktur data komposit seperti record.

- Navigasi Anggota: Atribut last adalah pointer yang sangat penting; ia menunjuk ke identifier terakhir yang dideklarasikan dalam blok tersebut di dalam tabel tab. Dari last inilah compiler bisa merunut balik semua variabel lokal atau field dari sebuah record melalui atribut link pada tabel tab.
- Parameter dan Frame: Khusus untuk fungsi dan prosedur, lpar menunjuk ke parameter terakhir, sementara psze (parameter size) dan vsze (variable size) digunakan untuk menghitung total ukuran activation record atau stack frame saat fungsi tersebut dipanggil.

1.5. Visitor Pattern dalam Traversal Semantik

Dalam perancangan compiler modern, Visitor Pattern adalah pola desain perilaku (behavioral design pattern) yang digunakan untuk memisahkan algoritma atau operasi dari struktur objek tempat algoritma tersebut bekerja. Dalam konteks Analisis Semantik, pola ini menjadi standar untuk menelusuri simpul-simpul pada Abstract Syntax Tree (AST).

A. Konsep Dasar Visitor Pattern

Pada tahap analisis semantik, compiler perlu melakukan berbagai operasi pada AST, seperti type checking, manajemen tabel simbol, dan penghitungan alamat memori. Jika logika-logika ini dimasukkan langsung ke dalam kelas simpul AST, kelas-kelas tersebut akan menjadi sangat besar dan kompleks.

Visitor Pattern merupakan salah satu design pattern OOP yang memecahkan masalah ini dengan cara membagi struktur program menjadi dua komponen:

- Element (AST Nodes): Tetap fokus pada representasi data. Setiap simpul hanya memiliki satu metode utama, biasanya dinamakan `accept(visitor)`.
- Visitor: Sebuah objek terpisah yang mendefinisikan operasi spesifik untuk setiap jenis simpul. Dengan cara ini, bisa ditambah fungsi baru seperti optimasi atau pembuatan kode tanpa harus mengubah kelas simpul AST-nya.

Hal ini menyebabkan program menjadi memiliki beberapa keuntungan:

- Modularitas: Logika pengecekan semantik untuk array terpisah rapi dari logika pengecekan blok prosedur (yang melibatkan `btabs`).
- Kemudahan Dekorasi: Visitor dapat dengan mudah melakukan anotasi pada setiap simpul AST untuk menghasilkan Decorated AST karena ia memiliki akses penuh terhadap atribut simpul tersebut.
- Extensibility: Jika dilanjutkan pada tahap code generation, hanya cukup dibuat visitor baru.

B. Mekanisme Double Dispatch

Kunci utama Visitor Pattern adalah mekanisme Double Dispatch. Saat metode `node.accept(visitor)` dipanggil:

- Program menentukan tipe simpul (node) secara dinamis
- Simpul tersebut memanggil balik metode spesifik pada visitor, misalnya `visit(this)`
- Visitor kemudian menjalankan logika analisis semantik yang sesuai untuk jenis simpul tersebut

C. Traversal Semantik

Visitor Pattern sangat efektif untuk melakukan Traversal Semantik. Dalam analisis semantik, urutan kunjungan simpul sangat krusial:

- Pre-order Traversal: Digunakan saat informasi harus mengalir ke bawah misalnya, mengirimkan informasi scope atau level leksikal dari induk ke anak.
- Post-order Traversal: Digunakan saat informasi harus dikumpulkan dari bawah ke atas misalnya, menentukan tipe data hasil ekspresi $a + b$ setelah tipe a dan b diketahui.

Dalam semantic analysis ini, setiap kali visitor mengunjungi simpul yang merepresentasikan blok seperti prosedur atau fungsi, visitor akan melakukan operasi pada stack tabel simbol seperti menambah level atau memperbarui pointer tabel.

1.6. Type Checking dan Konsistensi Semantik

A. Type Checking pada Simple Type

Type checking adalah proses inti dalam analisis semantik yang memastikan bahwa setiap operasi dalam program dilakukan pada tipe data yang kompatibel.

- Aturan Kompatibilitas: Dalam beberapa bahasa pemrograman seperti yang digunakan dalam tugas besar kali ini, tidak semua tipe data dapat berinteraksi. Contohnya assignment compatibility bahwa sebuah variabel hanya dapat menerima nilai yang tipenya sama atau kompatibel misalnya nilai integer sering kali diizinkan untuk diberikan ke variabel real melalui implicit type conversion. Lalu, operator compatibility yaitu operator aritmetika (+, -, *) memerlukan operand bertipe angka, sementara operator logika (and, or, not) memerlukan operand boolean.
- Type Inference: Compiler harus mampu menentukan tipe dari sebuah ekspresi kompleks. Misalnya, dalam ekspresi $(a + b) > c$, compiler pertama-tama menentukan tipe $(a + b)$. Jika a adalah integer dan b adalah real, maka hasilnya adalah real. Kemudian, hasil real tersebut dibandingkan dengan c menggunakan operator $>$, yang akhirnya menghasilkan tipe boolean.

B. Konsistensi Semantik

Konsistensi semantik memeriksa apakah logika alur program mematuhi aturan kontekstual bahasa yang tidak dapat ditangkap oleh grammar.

- Adanya dan keunikan deklarasi yaitu memastikan setiap variabel atau fungsi yang dipanggil sudah dideklarasikan sebelumnya dalam tabel

simbol serta memastikan tidak ada dua identifier dengan nama yang sama dalam satu scope yang sama.

- Validasi subprogram yaitu memastikan jumlah parameter yang dikirimkan saat pemanggilan fungsi sesuai dengan jumlah yang didefinisikan di btab. Selain itu, urutan dan tipe datanya juga harus cocok. Terakhir, dipastikan fungsi mengembalikan nilai dengan tipe yang benar dan setiap jalur eksekusi dalam fungsi memiliki return value.
- Kontrol struktur yaitu memastikan perintah seperti break atau continue hanya muncul di dalam blok perulangan (while, for, repeat). Selain itu, dipastikan juga ekspresi di dalam kondisi if atau while selalu bernilai boolean.

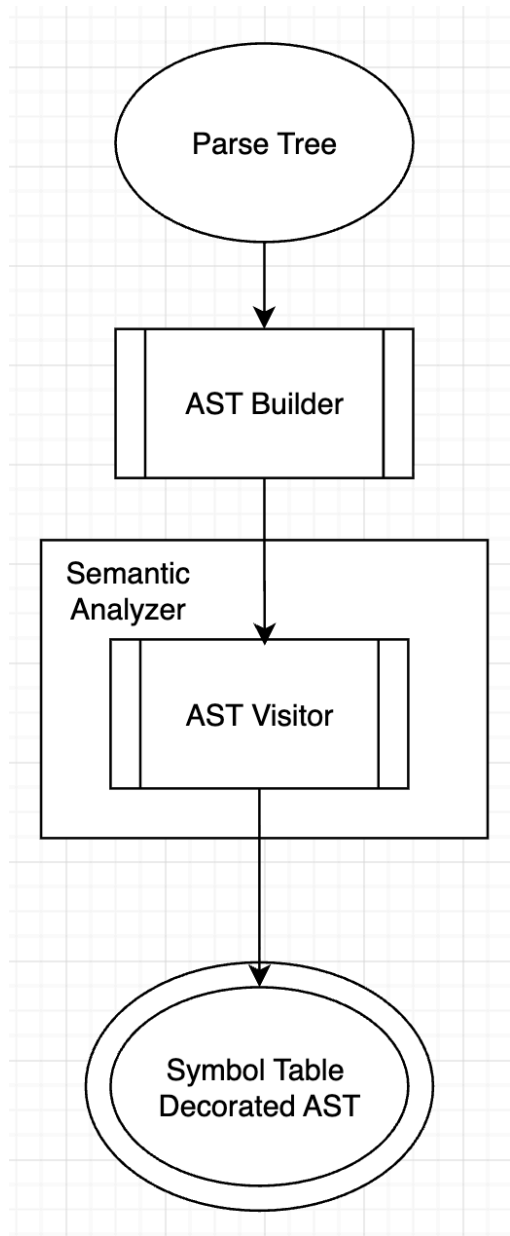
2. Perancangan dan Implementasi

2.1. Teknologi yang Digunakan

Program ini dibangun menggunakan bahasa C++ dengan memanfaatkan Standard Template Library (STL) seperti `std::vector`, `std::string`, dan `std::shared_ptr`. Program juga menggunakan makefile untuk mengotomatisasi proses kompilasi program. Alasan penggunaan bahasa C++ beserta STL untuk memaksimalkan performa dan menjamin keamanan memori, serta konfigurasi dengan makefile untuk memastikan siklus kompilasi yang otomatis, konsisten, dan efisien.

2.2. Alur Eksekusi Semantic Analyzer

Dalam eksekusi semantic analyzer akan menerima input berupa parse tree sebelum memasuki algoritma utama semantic analyzer seperti pengisian dari symbol table maka perlu dibuat sebuah bentuk sederhana dari parse tree yang disebut Abstract Syntax Tree. Isi dari abstract syntax tree ini akan berisi inti dari aturan produksi yang telah dibuat pada parse tree. Parse Tree akan disederhanakan menjadi aturan semantiknya yaitu 27 jenis node semantik. Setelah dibuat AST ini akan dilanjutkan lagi scan ke 2 dalam membentuk Decorated AST yaitu mengisi symbol table yang berisi table berupa reservasi identifier, konstanta, fungsi build in bahasa Arion dan variable. fungsi dan . Bahasa Arion ini menggunakan dua kali scan dalam menentukan aturan semantiknya yang bisa dilihat dari alur program dibawah ini.



Scan pertama dilakukan untuk membuat AST dan scan kedua dilakukan untuk membuat decorated AST dan mengisi symbol table.

2.3. Arsitektur Program

Semantic Analyzer dirancang menggunakan komponen-komponen sebagai berikut:

A. AST

Kelas dasar yang digunakan untuk merepresentasikan elemen dalam AST. Kelas ini di-extend menjadi turunan yaitu deklarasi, type, ekspresi dan statement yang lalu di-extend kembali menjadi elemen paling spesifik seperti const atau binary operand. Kelas ini memiliki atribut tambahan ObjectType untuk menentukan jenis objek dan DataType untuk menentukan tipe data yang terkait pada node. Struktur data ini dibangun menggunakan kelas ASTBuilder.

B. SymbolTable

Kelas dasar yang digunakan untuk merepresentasikan tabel-tabel yang digunakan untuk semantic analysis. SymbolTable terdiri dari atribut tab, atab, dan btab yang dibangun menggunakan struct serta fungsi helper yang digunakan untuk validasi, pencarian, dan penambahan simbol yang diintegrasikan dengan SemanticVisitor.

C. Visitor

Visitor Pattern digunakan untuk memisahkan logika semantic analysis dari struktur data AST. Konsep visitor diimplementasikan melalui kelas ASTVisitor dengan setiap node ASTNode memiliki metode accept() yang menerima objek visitor. Hal ini memungkinkan implementasi visitor yang berbeda untuk operasi yang berbeda pada tiap node. Implementasinya kemudian ada di kelas SemanticVisitor yang berfungsi sebagai proses utama semantic analysis yaitu menambahkan informasi semantic ke node, mengecek adanya kesalahan semantic, serta menambahkan dan mengecek simbol berdasarkan deklarasi AST.

D. Lain-lain

Komponen pembantu yang digunakan untuk mencetak struktur AST dan informasi table agar mudah dibaca melalui ASTPrinter serta SemanticAnalyzer sebagai kelas utama yang mengelola pipeline untuk mengintegrasikan seluruh komponen lalu mencetak dan mendekorasi output

2.4. Kelas SymbolTable

Kelas SymbolTable merupakan komponen utama dalam tahap semantic analysis. Kelas ini berfungsi sebagai penyimpanan seluruh informasi yang tersedia mengenai suatu identifer yang terdaftar di dalam kode program. Tabel ini diimplementasikan memiliki tiga table dalam bentuk struct yaitu tab sebagai tabel umum identifer, atab sebagai array table, dan btab sebagai block table. Detail atribut atau komponen dari tiap tabel disesuaikan dengan spesifikasi tugas besar.

```
struct TabEntry {
    std::string name;    // identifiers
    int link;            // indeks ke ident sebelumnya dalam scope sama
    ObjectType obj;      // jenis objek
    DataType type;       // tipe dasar
    int ref;             // indeks ke atab atau btab
    int nrm;             // 1 = normal, 0 = by-reference
    int lev;            // lexical level (0 = global, 1 = dalam
    prosedur, 2 = dalam prosedur di dalam prosedur, dst)
    long adr;            // nilai/alamat/offset
};

struct AtabEntry {
    DataType xtyp;       // tipe indeks
    DataType etyp;       // tipe elemen
    int eref;            // ref detail jika elemen komposit
    int low;             // batas bawah
    int high;            // batas atas
    int elsz;            // ukuran elemen
    int size;            // total ukuran array
};

struct BtabEntry {
    int last;            // indeks identifier terakhir di blok ini
    int lpar;            // indeks parameter terakhir
    int psze;            // total ukuran parameter
    int vsze;            // total ukuran variabel lokal
};
```

Ketiga struct itu kemudia diimplementasikan sebagai atribut bagian dari kelas SymbolTable

```
class SymbolTable
{
public:
    std::vector<TabEntry> tab;
    std::vector<AtabEntry> atab;
    std::vector<BtabEntry> btab;

    int tabTop = 0;
    int currentBlock = 0;
```

```

    int enter(std::string name, ObjectType kind, DataType type,
int lev);
    int lookup(std::string name, int lastIndex);

    SymbolTable() {
        tab.resize(33);
        atab.resize(1);
        btab.resize(1);
        initPredefined();
    }

    ~SymbolTable();

private:
    void initPredefined();
};

```

Selain memiliki atribut tiga tabel symbol, kelas SymbolTable juga memiliki atribut tabTop yang berfungsi untuk melacak indeks atau baris terakhir dari suatu blok yang terakhir ditulis dan atribut currentBlock yang berfungsi untuk melacak scope atau blok kode mana yang sedang diproses dalam semantic analysis.

SymbolTable juga memiliki fungsi helper yaitu

1. initPredefined() yang berfungsi untuk mengisi tabel dengan data bawaan yang sudah dikenali compiler sebelum melakukan semantic analysis kode program.
2. enter() yang berfungsi untuk mendaftarkan identifiier baru yang ditemukan di bagian deklarasi ke tab.
3. lookup() yang berfungsi untuk mencari keberadaan suatu identifiier dalam tabel ketika menemukannya di kode program.

Implementasi fungsi-fungsi tersebut di bawah ini

```

#include "SymbolTable.hpp"
#include <iostream>
#include <string>

void SymbolTable::initPredefined()
{
    currentBlock = 0;
    tabTop = 0;
    // index 0 : nullpointer untuk 'NULL';
    tab[0] = {"NULL", 0, ObjectType::TYPE, DataType::UNKNOWN, 0, 0, 0,
0};
    //          identifier, link, obj,          type,      ref, nrm,
level, address.
    tab[1] = {"INTEGER", 0, ObjectType::TYPE, DataType::INTEGER, 0, 1,
0, 4}; // size 4 byte
    tab[2] = {"REAL", 1, ObjectType::TYPE, DataType::REAL, 0, 1, 0, 8};
    // size 8 byte

```

```

    tab[3] = {"CHAR", 2, ObjectType::TYPE, DataType::CHAR, 0, 1, 0, 1};
// size 1 byte
    tab[4] = {"BOOLEAN", 3, ObjectType::TYPE, DataType::BOOLEAN, 0, 1,
0, 1}; // size 1 byte
    tab[5] = {"STRING", 4, ObjectType::TYPE, DataType::STRING, 0, 1, 0,
255}; // size 255 byte

    std::vector<std::string> reserveWord = {
        "AND", "NOT", "MOD", "DIV", "IF", "BEGIN", "CASE", "CONST",
        "DOWNT", "DO", "ELSE", "END", "FOR", "OF", "OR", "PROGRAM",
        "REPEAT", "THEN", "TO", "TYPE", "VAR", "UNTIL", "WHILE",
        "ARRAY", "RECORD", "FUNCTION", "PROCEDURE"};

    int currentLink = 5;
    int i = 6;
    for (const std::string &word : reserveWord) {
        if (i >= (int)tab.size()) tab.resize(tab.size() + 20);
        tab[i].name = word;
        tab[i].obj = ObjectType::RESERVE;
        tab[i].type = DataType::VOID;
        tab[i].link = currentLink;
        currentLink = i;
        i++;
    }

    // Set btab[0].last to point to the last reserved word FIRST
    tabTop = i - 1;
    btab[0].last = tabTop;

    // Now add predefined I/O procedures at level 0 (global)
    // enter() will use btab[0].last + 1 to find the next index
    enter("writeln", ObjectType::PROCEDURE, DataType::VOID, 0);
    enter("write", ObjectType::PROCEDURE, DataType::VOID, 0);
    enter("read", ObjectType::PROCEDURE, DataType::VOID, 0);
    enter("readln", ObjectType::PROCEDURE, DataType::VOID, 0);

    enter("True", ObjectType::CONSTANT, DataType::BOOLEAN, 0);
    enter("False", ObjectType::CONSTANT, DataType::BOOLEAN, 0);
};

int SymbolTable::lookup(std::string name, int lastIndex)
{
    int i = lastIndex;
    while (i > 0) {
        if (tab[i].name == name) return i;
        i = tab[i].link;
    }
    return 0;
}

int SymbolTable::enter(std::string name, ObjectType kind, DataType
type, int lev)
{
    // Cari indeks kosong berikutnya.
    tabTop++;
    int index = tabTop;

    // Jika kapasitas tabel hampir penuh, perbesar ukurannya
    if (index >= (int)tab.size()) {
        tab.resize(tab.size() + 20);
    }
}

```

```

// Masukkan data ke dalam tabel
tab[index].name = name;
tab[index].obj = kind;
tab[index].type = type;
tab[index].lev = lev;

tab[index].link = btab[currentBlock].last;
btab[currentBlock].last = index;

return index;
}

SymbolTable::~SymbolTable() = default;

```

Predefined identifier dalam tugas besar ini

Identifier	Keterangan
NULL	Representasi null pointer atau tipe tidak diketahui (berada di indeks 0).
INTEGER	Tipe data bilangan bulat (ukuran 4 byte).
REAL	Tipe data bilangan pecahan/desimal (ukuran 8 byte).
CHAR	Tipe data karakter tunggal (ukuran 1 byte).
BOOLEAN	Tipe data logika benar/salah (ukuran 1 byte).
STRING	Tipe data teks (ukuran maksimal 255 byte).
writeln	Prosedur standar untuk mencetak output dengan baris baru (newline).
write	Prosedur standar untuk mencetak output tanpa baris baru.
read	Prosedur standar untuk membaca input dari pengguna.
readln	Prosedur standar untuk membaca input dan menyerap baris baru.
TRUE	Nilai konstanta bawaan untuk logika benar (bertipe BOOLEAN).
FALSE	Nilai konstanta bawaan untuk logika salah (bertipe BOOLEAN).
AND, NOT, MOD, DIV	Operator logika dan aritmetika dasar.
IF, THEN, ELSE, CASE, OF	Percabangan dan kontrol alur logika program.

FOR, TO, DOWNT0, DO	Perulangan (looping) dengan perhitungan angka.
WHILE, REPEAT, UNTIL	Perulangan berdasarkan evaluasi kondisi (boolean).
PROGRAM, BEGIN, END	Penanda struktur blok utama program.
VAR, CONST, TYPE	Penanda bagian deklarasi variabel, konstanta, dan tipe data.
ARRAY, RECORD	Penanda deklarasi tipe data terstruktur/komposit.
FUNCTION, PROCEDURE	Penanda deklarasi subprogram.

2.5. Kelas ASTNode dan ASTBuilder

Kelas `ASTNode` merupakan representasi Abstract Syntax Tree dan Decorated Syntax Tree dalam program. Kelas ini merepresentasikan simpul atau node yang terdapat dalam AST. Kelas ini memiliki atribut:

1. `ASTNodeType nodeType` yang merupakan enum jenis node dari AST untuk mengetahui jenis node
2. `line` dan `column` untuk mengetahui posisi jika semantic analysis menghasilkan error sehingga mudah untuk di-debug
3. `scopeLevel` untuk mengetahui scope dari node yang sedang ditelusuri
4. `semanticErrors` sebagai vector untuk menyimpan pesan semantic error program

Kelas ini memiliki method untuk menerapkan visitor pattern menggunakan `accept(ASTVisitor* visitor)` sehingga memungkinkan pemisahan antara struktur data dengan algoritma pengecekan semantik.

```
class ASTNode {
public:
    ASTNodeType nodeType;
    int scopeLevel = -1;
    std::vector<std::string> semanticErrors;

    int line;
    int column;

    ASTNode(ASTNodeType t) : nodeType(t) {}
    virtual ~ASTNode() = default;
```

```
virtual void accept(ASTVisitor* visitor) = 0;
};
```

Kelas ini menerapkan konsep inheritance untuk merepresentasikan berbagai komponen bahasa pemrograman Arion. Alasan konsep inheritance digunakan untuk memungkinkan adanya polimorfisme, code reusability, dan penerapan visitor pattern. Secara garis besar, ASTNode dibagi menjadi lima kelas utama:

1. ProgramNode dan BlockNode

Mengatur kerangka utama program yang terdiri dari bagian deklarasi dan bagian statement

```
class BlockNode : public ASTNode {
public:
    std::vector<std::shared_ptr<DeclarationNode>>
    declarations;
    std::shared_ptr<CompoundStatementNode> statements;

    BlockNode() : ASTNode(ASTNodeType::BlockNode) {}
    void accept(ASTVisitor* visitor) override {
        visitor->visit(this); }
};

class ProgramNode : public ASTNode {
public:
    std::string name;
    std::vector<std::shared_ptr<DeclarationNode>>
    declarations;
    std::shared_ptr<CompoundStatementNode> statements;

    ProgramNode(const std::string& n) :
    ASTNode(ASTNodeType::Program), name(n) {}
    void accept(ASTVisitor* visitor) override {
        visitor->visit(this); }
};
```

2. DeclarationNode

Menangani deklarasi identifer di awal kode pemrograman meliputi ConstDeclarationNode, VarDeclarationNode, TypeDeclarationNode, ProcedureDeclarationNode, dan FunctionDeclarationNode.

```
class DeclarationNode : public ASTNode {
public:
    int tabIndex = 0;
    DeclarationNode(ASTNodeType t) : ASTNode(t) {}
};
```

```

// CONST: Nama = Nilai
class ConstDeclarationNode : public DeclarationNode {
public:
    std::string name;
    std::shared_ptr<ExpressionNode> value;

    ConstDeclarationNode(const std::string& n,
std::shared_ptr<ExpressionNode> val)
        : DeclarationNode (ASTNodeType::ConstDecl), name(n),
value(val) {}

    void accept(ASTVisitor* visitor) override {
visitor->visit(this); }
};

// VAR: Nama = Tipe
class VarDeclarationNode : public DeclarationNode {
public:
    std::string name;
    std::shared_ptr<TypeNode> typeDefinition;

    VarDeclarationNode(const std::string& n,
std::shared_ptr<TypeNode> typeDef)
        : DeclarationNode (ASTNodeType::VarDecl), name(n),
typeDefinition(typeDef) {}

    void accept(ASTVisitor* visitor) override {
visitor->visit(this); }
};

// TYPE: Nama = Struktur Tipe
class TypeDeclarationNode : public DeclarationNode {
public:
    std::string name;
    std::shared_ptr<TypeNode> typeDefinition;

    TypeDeclarationNode(const std::string& n,
std::shared_ptr<TypeNode> typeDef)
        : DeclarationNode (ASTNodeType::TypeDecl), name(n),
typeDefinition(typeDef) {}

    void accept(ASTVisitor* visitor) override {
visitor->visit(this); }
};

// PROCEDURE: Nama = Parameter + Block
class ProcedureDeclarationNode : public DeclarationNode {
public:
    std::string name;
    std::vector<std::shared_ptr<VarDeclarationNode>>
parameters;
    std::shared_ptr<BlockNode> body;
    int localVariablesSize = 0;

    ProcedureDeclarationNode(const std::string& n)

```

```

        : DeclarationNode (ASTNodeType::ProcDecl), name(n)
    {}

    void accept(ASTVisitor* visitor) override {
        visitor->visit(this); }
};

// FUNCTION: Nama = Parameter + Return Type + Block
class FunctionDeclarationNode : public DeclarationNode {
public:
    std::string name;
    std::vector<std::shared_ptr<VarDeclarationNode>>
parameters;
    std::shared_ptr<TypeNode> returnType;
    std::shared_ptr<BlockNode> body;
    int localVariablesSize = 0;

    FunctionDeclarationNode(const std::string& n,
std::shared_ptr<TypeNode> retType)
        : DeclarationNode (ASTNodeType::FuncDecl), name(n),
returnType(retType) {}

    void accept(ASTVisitor* visitor) override {
        visitor->visit(this); }
};

```

3. TypeNode

Menangani representasi tipe data mulai dari primitif hingga yang terstruktur meliputi SimpleTypeNode, ArrayTypeNode, RecordTypeNode, dan RangeTypeNode. Node ini memiliki atribut resolvedType yang diisi pada tahap semantik.

```

class TypeNode : public ASTNode {
public:
    DataType resolvedType = DataType::UNKNOWN;
    int resolvedRef = 0;
    TypeNode(ASTNodeType t) : ASTNode(t) {}
};

// Tipe Dasar (misal: "integer", "real", "boolean", atau
nama alias custom)
class SimpleTypeNode : public TypeNode {
public:
    std::string name;

    SimpleTypeNode(const std::string& n) :
TypeNode(ASTNodeType::SimpleType), name(n) {}
    void accept(ASTVisitor* visitor) override {
        visitor->visit(this); }
};

// Tipe Array (misal: array [1..10] of integer)
class ArrayTypeNode : public TypeNode {

```

```

public:
    std::shared_ptr<TypeNode> indexType;
    std::shared_ptr<TypeNode> elementType;

    ArrayTypeNode(std::shared_ptr<TypeNode> idxType,
std::shared_ptr<TypeNode> elemType)
        : TypeNode(ASTNodeType::ArrayType),
indexType(idxType), elementType(elemType) {}

    void accept(ASTVisitor* visitor) override {
visitor->visit(this); }
};

// Tipe Record (misal: record x: integer; y: real; end)
class RecordTypeNode : public TypeNode {
public:
    std::vector<std::shared_ptr<VarDeclarationNode>>
fields;

    RecordTypeNode() : TypeNode(ASTNodeType::RecordType) {}
    void accept(ASTVisitor* visitor) override {
visitor->visit(this); }
};

// Tipe Enumerated (misal: (Senin, Selasa, Rabu))
class EnumeratedTypeNode : public TypeNode {
public:
    std::vector<std::string> elements;
    EnumeratedTypeNode() :
TypeNode(ASTNodeType::EnumeratedType) {}
    void accept(ASTVisitor* visitor) override {
visitor->visit(this); }
};

// Tipe Range (misal: 1..100)
class RangeTypeNode : public TypeNode {
public:
    std::shared_ptr<ExpressionNode> lowBound;
    std::shared_ptr<ExpressionNode> highBound;
    DataType baseType = DataType::UNKNOWN;

    RangeTypeNode(std::shared_ptr<ExpressionNode> low,
std::shared_ptr<ExpressionNode> high)
        : TypeNode(ASTNodeType::RangeType), lowBound(low),
highBound(high) {}
    void accept(ASTVisitor* visitor) override {
visitor->visit(this); }
};

```

4. ExpressionNode

Menangani segala hal yang menghasilkan nilai , seperti nilai literal seperti angka atau teks, pemanggilan variabel, operasi

aritmetika/logika (BinaryOp, UnaryOp), hingga pemanggilan fungsi (FuncCall).

```
class ExpressionNode : public ASTNode {
public:
    DataType evaluatedType = DataType::UNKNOWN;
    int evaluatedRef = 0;

    ExpressionNode(ASTNodeType t) : ASTNode(t) {}
};

class IntegerLiteralNode : public ExpressionNode {
public:
    int value;
    IntegerLiteralNode(int val) :
    ExpressionNode(ASTNodeType::IntLiteral), value(val) {}
    void accept(ASTVisitor* visitor) override {
        visitor->visit(this); }
};

class RealLiteralNode : public ExpressionNode {
public:
    double value;
    RealLiteralNode(double val) :
    ExpressionNode(ASTNodeType::RealLiteral), value(val) {}
    void accept(ASTVisitor* visitor) override {
        visitor->visit(this); }
};

class StringLiteralNode : public ExpressionNode {
public:
    std::string value;
    StringLiteralNode(const std::string& val) :
    ExpressionNode(ASTNodeType::StringLiteral), value(val) {}
    void accept(ASTVisitor* visitor) override {
        visitor->visit(this); }
};

class CharLiteralNode : public ExpressionNode {
public:
    char value;
    CharLiteralNode(char val) :
    ExpressionNode(ASTNodeType::CharLiteral), value(val) {}
    void accept(ASTVisitor* visitor) override {
        visitor->visit(this); }
};

class BooleanLiteralNode : public ExpressionNode {
public:
    bool value;
    BooleanLiteralNode(bool val) :
    ExpressionNode(ASTNodeType::BoolLiteral), value(val) {}
    void accept(ASTVisitor* visitor) override {
        visitor->visit(this); }
};
```

```

class VarAccessNode : public ExpressionNode {
public:
    std::string name;
    bool isConstant = false;
    int tabIndex = 0;

    VarAccessNode(const std::string& n) :
    ExpressionNode(ASTNodeType::VarAccess), name(n) {}
    void accept(ASTVisitor* visitor) override {
    visitor->visit(this); }
};

class ArrayAccessNode : public ExpressionNode {
public:
    std::shared_ptr<ExpressionNode> target;
    std::shared_ptr<ExpressionNode> index;

    ArrayAccessNode(std::shared_ptr<ExpressionNode> tgt,
    std::shared_ptr<ExpressionNode> idx)
        : ExpressionNode(ASTNodeType::ArrayAccess),
    target(tgt), index(idx) {}
    void accept(ASTVisitor* visitor) override {
    visitor->visit(this); }
};

class FieldAccessNode : public ExpressionNode {
public:
    std::shared_ptr<ExpressionNode> target;
    std::string fieldName;
    int tabIndex = 0;

    FieldAccessNode(std::shared_ptr<ExpressionNode> tgt,
    const std::string& field)
        : ExpressionNode(ASTNodeType::FieldAccess),
    target(tgt), fieldName(field) {}
    void accept(ASTVisitor* visitor) override {
    visitor->visit(this); }
};

class BinaryOpNode : public ExpressionNode {
public:
    std::string op;
    std::shared_ptr<ExpressionNode> left;
    std::shared_ptr<ExpressionNode> right;

    BinaryOpNode(const std::string& o,
    std::shared_ptr<ExpressionNode> l,
    std::shared_ptr<ExpressionNode> r)
        : ExpressionNode(ASTNodeType::BinaryOp), op(o),
    left(l), right(r) {}

    void accept(ASTVisitor* visitor) override {
    visitor->visit(this); }
};

class UnaryOpNode : public ExpressionNode {

```

```

public:
    std::string op;
    std::shared_ptr<ExpressionNode> operand;

    UnaryOpNode(const std::string& o,
std::shared_ptr<ExpressionNode> expr)
        : ExpressionNode(ASTNodeType::UnaryOp), op(o),
operand(expr) {}

    void accept(ASTVisitor* visitor) override {
visitor->visit(this); }
};

class FunctionCallNode : public ExpressionNode {
public:
    std::string name;
    std::vector<std::shared_ptr<ExpressionNode>> args;
    int tabIndex = 0;

    FunctionCallNode(const std::string& n) :
ExpressionNode(ASTNodeType::FuncCall), name(n) {}

    void accept(ASTVisitor* visitor) override {
visitor->visit(this); }
};

```

5. StatementNode

Menangani instruksi eksekusi kode, seperti percabangan (IfStatement, CaseStatement), perulangan (WhileLoop, ForLoop, RepeatUntil), penugasan nilai (AssignmentStatement), hingga blok statemen gabungan (CompoundStatement).

```

class StatementNode : public ASTNode {
public:
    StatementNode(ASTNodeType type) : ASTNode(type) {}
};

// COMPOUND STATEMENT (begin ... end)
class CompoundStatementNode : public StatementNode {
public:
    std::vector<std::shared_ptr<StatementNode>> statements;

    CompoundStatementNode() :
StatementNode(ASTNodeType::CompoundStatement) {}
    void accept(ASTVisitor* visitor) override {
visitor->visit(this); }
};

// ASSIGNMENT STATEMENT (:=)
class AssignmentStatementNode : public StatementNode {
public:
    std::shared_ptr<ExpressionNode> target;

```



```

        std::shared_ptr<ExpressionNode> value;

        AssignmentStatementNode(std::shared_ptr<ExpressionNode>
tgt, std::shared_ptr<ExpressionNode> val)
            : StatementNode(ASTNodeType::AssignmentStatement),
target(tgt), value(val) {}
        void accept(ASTVisitor* visitor) override {
visitor->visit(this); }
};

// IF STATEMENT (if ... then ... else)
class IfStatementNode : public StatementNode {
public:
    std::shared_ptr<ExpressionNode> condition;
    std::shared_ptr<StatementNode> thenBranch;
    std::shared_ptr<StatementNode> elseBranch;

    IfStatementNode(std::shared_ptr<ExpressionNode> cond,
std::shared_ptr<StatementNode> thenBr,
std::shared_ptr<StatementNode> elseBr = nullptr)
        : StatementNode(ASTNodeType::IfStatement),
condition(cond), thenBranch(thenBr), elseBranch(elseBr) {}
    void accept(ASTVisitor* visitor) override {
visitor->visit(this); }
};

// CASE STATEMENT (case ... of ... end)
class CaseStatementNode : public StatementNode {
public:
    std::shared_ptr<ExpressionNode> expression;

    std::vector<std::pair<std::vector<std::shared_ptr<Expressio
nNode>>, std::shared_ptr<StatementNode>>> cases;
    std::shared_ptr<StatementNode> elseBranch; // Nullable

    CaseStatementNode(std::shared_ptr<ExpressionNode> expr)
        : StatementNode(ASTNodeType::CaseStatement),
expression(expr), elseBranch(nullptr) {}
    void accept(ASTVisitor* visitor) override {
visitor->visit(this); }
};

// WHILE LOOP (while ... do)
class WhileLoopNode : public StatementNode {
public:
    std::shared_ptr<ExpressionNode> condition;
    std::shared_ptr<StatementNode> body;

    WhileLoopNode(std::shared_ptr<ExpressionNode> cond,
std::shared_ptr<StatementNode> b)
        : StatementNode(ASTNodeType::WhileStatement),
condition(cond), body(b) {}
    void accept(ASTVisitor* visitor) override {
visitor->visit(this); }
};

```

```

// FOR LOOP (for ... to/downto ... do)
class ForLoopNode : public StatementNode {
public:
    std::string counterVar;
    std::shared_ptr<ExpressionNode> startValue;
    std::shared_ptr<ExpressionNode> endValue;
    bool isDownTo; // false untuk 'to', true untuk 'downto'
    std::shared_ptr<StatementNode> body;

    ForLoopNode(const std::string& var,
std::shared_ptr<ExpressionNode> start,
std::shared_ptr<ExpressionNode> end, bool downTo,
std::shared_ptr<StatementNode> b)
        : StatementNode(ASTNodeType::ForStatement),
counterVar(var), startValue(start), endValue(end),
isDownTo(downTo), body(b) {}
    void accept(ASTVisitor* visitor) override {
visitor->visit(this); }
};

// REPEAT UNTIL LOOP (repeat ... until)
class RepeatUntilNode : public StatementNode {
public:
    std::vector<std::shared_ptr<StatementNode>> body;
    std::shared_ptr<ExpressionNode> condition;

    RepeatUntilNode(std::shared_ptr<ExpressionNode> cond)
        : StatementNode(ASTNodeType::RepeatUntil),
condition(cond) {}
    void accept(ASTVisitor* visitor) override {
visitor->visit(this); }
};

// PROCEDURE CALL STATEMENT
class ProcedureCallNode : public StatementNode {
public:
    std::string name;
    std::vector<std::shared_ptr<ExpressionNode>> args;
    int tabIndex = 0;

    ProcedureCallNode(const std::string& n)
        : StatementNode(ASTNodeType::ProcedureCall),
name(n) {}
    void accept(ASTVisitor* visitor) override {
visitor->visit(this); }
};

```

Kelas **ASTBuilder** digunakan untuk melakukan translasi dari **TreeNode** (pohon sintaksis mentah yang dihasilkan oleh parser) menjadi pohon **ASTNode**.

Proses pembangunan dimulai dari akar pohon melalui fungsi **build()** yang memanggil **buildProgram()**. Dari fungsi tersebut, kelas ini akan

secara rekursif memanggil fungsi spesifik lainnya (seperti buildBlock, buildDeclarationPart, buildStatement) menuruni struktur anak-anaknya (children) hingga mencapai node daun (literal atau identifer).

Proses pembangunannya menggunakan semantic action berdasarkan Syntax-Directed Translation Scheme sebagai berikut

Production Rule	Semantic Rule
<code><program> → <program-header> <declaration-part> <compound-statement></code>	<pre> program_header.name = identifier.lexeme program.node = new ProgramNode(program_header.name, declaration_part.node_list, compound_statement.node) </pre>
<code><block> → <declaration-part> <compound-statement></code>	<pre> block.node = new BlockNode(declaration_part.node_list, compound_statement.node) </pre>
<code><declaration-part> → <const-decl-part> <type-decl-part> <var-decl-part> <proc-decl>* <func-decl>*</code>	<pre> declaration_part.node_list = const_decl_part.node_list type_decl_part.node_list var_decl_part.node_list {procedure_declaration.node} {function_declaration.node} </pre>
<code><const-decl-list> → IDENT = <constant> ; <const-decl-list> ε</code>	<pre> const_decl_list.node_list.append(new ConstDeclarationNode (IDENT.lexeme, constant.node)) </pre>
<code><type-decl-list> → IDENT = <type> ; <type-decl-list> ε</code>	<pre> type_decl_list.node_list.append(new TypeDeclarationNode (IDENT.lexeme, type.node)) </pre>
<code><var-declaration> → <identifier-list> : <type> ;</code>	<pre> var_declaration.node_list.append(new VarDeclarationNode(id, type.node)) </pre>
<code><procedure-declaration> → procedure IDENT (<formal-parameter-list>) ; <block> ;</code>	<pre> procedure_declaration.node = new ProcedureDeclarationNode(IDENT.lexeme, formal_parameter_list.node_list, block.node) </pre>
<code><function-declaration> → function IDENT₁ (<formal-parameter-list>) : IDENT₂ ; <block> ;</code>	<pre> function_declaration.node = new FunctionDeclarationNode(IDENT₁.lexeme, new SimpleTypeNode (IDENT₂.lexeme), formal_parameter_list.node_list, block.node) </pre>
<code><parameter-group> → <identifier-list> : IDENT</code>	<pre> formal_parameter_list.node_list.append(new VarDeclarationNode(id, new </pre>

<pre> <formal-parameter-list> → <parameter-group> (; <parameter-group>)*</pre>	<pre> SimpleTypeNode(IDENT.lexeme)))</pre>
<pre> <type> → IDENT <record-type> <array-type> <range-type> <enumerated-type></pre>	<pre> type.node = new SimpleTypeNode(IDENT.lexeme) type.node = record_type.node type.node = array_type.node type.node = range_type.node type.node = enumerated_type.node</pre>
<pre> <record-type> → record <field-list> end <field-list> → <field-part> (; <field-part>)* <field-part> → <identifier-list> : <type></pre>	<pre> field_list.node_list.append(new VarDeclarationNode(id, type.node)) record_type.node = new RecordTypeNode(field_list.node_list)</pre>
<pre> <index-type> → <range-type> IDENT <array-type> → array [<index-type>] of <type></pre>	<pre> index_type.node = range_type.node U new SimpleTypeNode(IDENT.lexeme) array_type.node = new ArrayTypeNode(index_type.node, type.node)</pre>
<pre> <range-type> → <constant>₁ .. <constant>₂</pre>	<pre> range_type.node = new RangeTypeNode(constant₁.node, constant₂.node)</pre>
<pre> <enumerated-type> → (<identifier-list>)</pre>	<pre> enumerated_type.node = new EnumeratedTypeNode({IDENT.lexeme})</pre>
<pre> <simple-expression> → (+ -)? <term> (<additive-operator> <term>)*</pre>	<pre> base_node = is_negative ? new UnaryOpNode("-", term₁.node) : term₁.node base_node = new BinaryOpNode(op.lexeme, base_node, term_next.node) simple_expression.node = base_node</pre>
<pre> <term> → <factor> (<multiplicative-operator> <factor>)*</pre>	<pre> base_node = factor₁.node base_node = new BinaryOpNode(op.lexeme, base_node, factor_next.node) term.node = base_node</pre>
<pre> <factor> → (not -)? ((<expression>) <constant> <variable> <procedure-call>)</pre>	<pre> core_node = expression.node constant.node variable.node procedure_call.node factor.node = has_unary ? new UnaryOpNode(unary.lexeme, core_node) : core_node</pre>
<pre> <variable> → IDENT <component-variable>* <component-variable> → [<index-list>] . IDENT</pre>	<pre> current_access = new VarAccessNode(IDENT.lexeme) new ArrayAccessNode(current_access, index.node) new FieldAccessNode(current_access, field_IDENT.lexeme)</pre>

<pre><index-list> → <expression> (, <expression>)*</pre>	<pre>variable.node = current_access</pre>
<pre><function-call> → IDENT (<parameter-list>) <parameter-list> → <expression> (, <expression>)*</pre>	<pre>function_call.node = new FunctionCallNode(IDENT.lexeme, {expression.node})</pre>
<pre><constant> → (+ -)? (INTCON REALCON STRING CHARCON IDENT)</pre>	<pre>literal_node = new IntegerLiteralNode(INTCON.value) new RealLiteralNode(REALCON.value) new StringLiteralNode(STRING.value) new CharLiteralNode(CHARCON.value) new VarAccessNode(IDENT.lexeme) constant.node = has_sign ? new UnaryOpNode(sign.lexeme, literal_node) : literal_node</pre>
<pre><statement> → <assignment-statement> <compound-statement> <if-statement> <case-statement> <while-statement> <for-statement> <repeat-statement> <procedure-call> ε</pre>	<pre>statement.node = assignment_statement.node compound_statement.node if_statement.node case_statement.node while_statement.node for_statement.node repeat_statement.node procedure_call.node nullptr</pre>
<pre><compound-statement> → begin <statement-list> end <statement-list> → <statement> (; <statement>)*</pre>	<pre>compound_statement.node = new CompoundStatementNode({statement.node})</pre>
<pre><assignment-statement> → <variable> := <expression></pre>	<pre>assignment_statement.node = new AssignmentStatementNode(variable.node , expression.node)</pre>
<pre><if-statement> → if <expression> then <statement>₁ (else <statement>₂)?</pre>	<pre>if_statement.node = new IfStatementNode(expression.node, statement₁.node, statement₂.node)</pre>
<pre><case-statement> → case <expression> of <case-block-list> end <case-block-list> → <case-block> (; <case-block>)* <case-block> → <constant> (, <constant>)* : <statement></pre>	<pre>case_statement.node = new CaseStatementNode(expression.node, { ({constant.node}, statement.node) })</pre>
<pre><while-statement> → while <expression> do <statement></pre>	<pre>while_statement.node = new WhileLoopNode(expression.node, statement.node)</pre>
<pre><for-statement> → for IDENT := <expression>₁ (to downto) <expression>₂ do <statement></pre>	<pre>for_statement.node = new ForLoopNode(IDENT.lexeme, expression₁.node, expression₂.node, is_downto, statement.node)</pre>

<pre><repeat-statement> → repeat <statement-list> until <expression></pre>	<pre>repeat_statement.node = new RepeatUntilNode(expression.node, {statement.node})</pre>
<pre><procedure-call> → IDENT (<parameter-list>) <parameter-list> → <expression> (, <expression>)*</pre>	<pre>procedure_call.node = new ProcedureCallNode(IDENT.lexeme, {expression.node})</pre>

Hasil Akhir dari ASTBuilder ini akan menghasilkan AST yang sudah disesuaikan untuk kebutuhan semantic analysis, namun atribut semantic-nya masih kosong.

Implementasi ASTBuilder ada di repository di file ASTBuilder.hpp dan ASTBuilder.cpp

2.6. Perancangan Semantic Analyzer dengan Semantic Visitor

Setelah konstruksi AST dan Symbol Table, dilakukan proses semantic analysis untuk memvalidasi kebenaran logika dan makna dari kode program. Implementasi kali ini menerapkan Visitor Design Pattern yang dirancang menggunakan interface ASTVisitor dan implementasi algoritmanya di SemanticVisitor.

Kelas ASTVisitor berfungsi sebagai interface yang mendefinisikan abstract method visit() untuk setiap jenis node di ASTNode. Pendekatan ini digunakan untuk memisahkan struktur data pohon dan algoritma operasional sehingga algoritma mudah dikembangkan.

```
class ASTVisitor {
public:
    virtual ~ASTVisitor() = default;

    virtual void visit(ProgramNode* node) = 0;
    virtual void visit(BlockNode* node) = 0;

    virtual void visit(SimpleTypeNode* node) = 0;
    .....// Disingkat agar tidak memenuhi laporan
    virtual void visit(EmptyStatementNode* node) = 0;
};
```

Implementasi interface ASTVisitor dilakukan oleh kelas SemanticVisitor. Kelas ini melakukan penelusuran pohon AST secara Depth-First Search secara Top-Down dan Bottom-Up untuk menjalankan fungsi-fungsi berikut:

1. Manajemen Scope dan Symbol Table

Untuk melacak seluruh deklarasi variabel, prosedur, dan fungsi, SemanticVisitor terintegrasi erat dengan kelas SymbolTable. Integrasi ini memastikan dua mekanisme kelola ruang lingkup berjalan dengan baik, yaitu sebagai berikut:

- Ketika visitor mengevaluasi ProcedureDeclarationNode atau FunctionDeclarationNode, SemanticVisitor akan secara otomatis membuat blok ruang lingkup baru (BtabEntry). Proses ini diikuti dengan menaikkan level kedalaman (currentLevel) dan memindahkan fokus pencarian ke blok yang baru. Setelah seluruh isi fungsi selesai diproses, fokus akan dikembalikan ke blok induknya (pop scope).
- Setiap kali menemukan node deklarasi baru, visitor mendaftarkannya melalui fungsi symbolTable.enter(). Apabila fungsi tersebut mendeteksi adanya duplikasi nama dan mengembalikan nilai -1, SemanticVisitor akan langsung mencatatnya sebagai error karena identifikasi tersebut sudah ada di ruang lingkup yang sama.

2. Type Checking

Selain melacak deklarasi, SemanticVisitor juga bertanggung jawab memastikan bahwa setiap operasi di dalam kode valid secara logika tipe data. Pemeriksaan ini diterapkan dalam beberapa skenario operasional, contohnya:

- Penelusuran pada BinaryOpNode dilakukan secara bottom-up. Visitor mengevaluasi tipe data dari simpul anak terlebih dahulu, lalu memanggil resolveBinaryType() untuk menyimpulkan tipe hasil akhirnya (misalnya, penjumlahan INTEGER dan REAL dipastikan akan menghasilkan REAL).
- Pada instruksi percabangan (IfStatementNode) dan perulangan (WhileLoopNode), SemanticVisitor mengawasi agar ekspresi syaratnya selalu dievaluasi dan mengembalikan tipe BOOLEAN.
- Pada pengisian nilai (AssignmentStatementNode), visitor memanfaatkan fungsi isCompatible() untuk memastikan bahwa tipe data di ruas kanan benar-benar dapat ditampung oleh variabel target di ruas kiri.

3. Proteksi Semantik Tambahan

SemanticVisitor menyimpan konteks khusus selama masa penelusuran AST. Hal ini bertujuan untuk mendeteksi pelanggaran tata bahasa yang lebih spesifik, seperti:

- Ketika mengevaluasi operasi penugasan, sistem akan memeriksa atribut ObjectType dari variabel target. Jika target tersebut terdaftar sebagai CONSTANT, maka upaya perubahan nilainya akan langsung ditolak.
- Perlindungan Variabel Pencacah (Counter Loop): Visitor menggunakan struktur activeLoopCounters untuk mengingat nama variabel pencacah yang sedang dipakai oleh perulangan FOR. Apabila di dalam tubuh perulangan terdapat instruksi yang memodifikasi nilai variabel ini, sistem akan memicu error modifikasi terlarang.

4. Perhitungan Alokasi Memori

Setiap kali visitor selesai mengevaluasi sebuah blok program baik global maupun subprogram, visitor akan melakukan iterasi pada Tabel Simbol untuk menghitung total memori parameter dari program dan total memori variabel program.

5. Manajemen Penanganan Kesalahan

SemanticVisitor juga dirancang untuk menangani kesalahan dengan mengumpulkan temuan error lalu mencetaknya di akhir proses dibandingkan langsung menghentikan proses semantic analysis secara paksa

Adapun Type Compability Rule yang digunakan pada SemanticAnalyzer ini sebagai berikut:

1. General Compability

Dua buah tipe data (T1 dan T2) dianggap saling kompatibel (compatible) secara dasar jika memenuhi salah satu dari kondisi berikut:

- Identitas Tipe: T1 dan T2 adalah tipe data yang sama persis (misal: INTEGER dengan INTEGER, atau BOOLEAN dengan BOOLEAN).

- Relasi Subrange: Salah satu tipe adalah SUBRANGE dan tipe lainnya adalah tipe dasar pembentuknya, yaitu INTEGER (atau kedua tipe sama-sama SUBRANGE). Di dalam sistem, tipe SUBRANGE dievaluasi secara dinamis agar kompatibel penuh dengan INTEGER.
- Tipe ARRAY kompatibel jika memiliki jumlah dimensi, rentang indeks, dan tipe elemen yang identik. Tipe RECORD kompatibel jika merujuk pada definisi struktur yang sama.

2. Assignment Compatibility

Sebuah nilai atau ekspresi dengan tipe T2 dapat di-assign ke dalam variabel target dengan tipe T1 jika memenuhi salah satu dari kondisi berikut:

- T1 dan T2 memiliki tipe data yang sama persis.
- Implicit Conversion: T1 adalah REAL dan T2 adalah INTEGER (Nilai bilangan bulat dapat dimasukkan ke dalam variabel desimal).
- Promosi Karakter: T1 adalah STRING dan T2 adalah CHAR (Satu karakter tunggal dapat dimasukkan ke dalam variabel kumpulan karakter/string).
- Subrange Assignment: T1 adalah SUBRANGE dan T2 adalah INTEGER, ataupun sebaliknya (T1 adalah INTEGER dan T2 adalah SUBRANGE).

3. Operator Compability

Kompatibilitas tipe dalam evaluasi ekspresi yang menggunakan operator (BinaryOpNode dan UnaryOpNode) ditentukan berdasarkan jenis operatornya:

- Operator Aritmetika Dasar (+, -, *):
Kedua operan harus berupa numerik (INTEGER, REAL, atau SUBRANGE). Hasil: REAL jika salah satu operan bernilai REAL; jika tidak, hasilnya INTEGER.
- Operator Pembagian Pecahan (/):
Kedua operan numerik. Hasil secara mutlak selalu REAL, meskipun kedua operannya berupa INTEGER.
- Operator Pembagian Bulat & Modulo (div, mod):

Kedua operan mutlak harus bertipe INTEGER. Jika diberikan REAL, sistem akan memicu error. Hasil selalu INTEGER.

- Operator Relasional/Perbandingan (=, ==, <>, <, <=, >, >=):
Kedua operan harus memiliki tipe dasar yang saling kompatibel dan selalu menghasilkan boolean
- Operator Tunggal:
Tanda minus/plus (-, +) operan mutlak harus numerik (INTEGER/REAL). Tanda negasi (not) operan mutlak harus BOOLEAN.
- Operator Logika (and, or):
Kedua operan secara mutlak harus bertipe BOOLEAN dan selalu menghasilkan boolean

4. Array dan Statement Compability

Selain penugasan dan operator, visitor juga memvalidasi ketat tipe data pada struktur kontrol sintaksis:

- Kondisi Percabangan dan Perulangan (If, While, Repeat-Until)
Ekspresi syarat yang dievaluasi harus menghasilkan tipe BOOLEAN. Jika tipe lain ditemukan (misal INTEGER), sistem akan memicu error semantik.
- Perulangan FOR
Variabel pencacah (counter), nilai awal (start value), dan nilai akhir (end value) ketiganya harus bertipe INTEGER.
- Akses Indeks Array
Tipe ekspresi yang dilewatkan di dalam tanda kurung siku [] (passed index type) harus assignment-compatible dengan tipe indeks array yang dideklarasikan.

5. Parameter Compability

Pada pemanggilan prosedur (ProcedureCallNode) maupun fungsi (FunctionCallNode), semantic analysis memastikan kesesuaian antara argumen dan parameter yaitu jumlah argumen yang dikirimkan pengguna harus sama persis dengan jumlah parameter yang dideklarasikan oleh blok subprogram terkait serta jumlah argumen yang dikirimkan pengguna harus sama persis dengan jumlah parameter yang dideklarasikan oleh blok subprogram terkait.

6. Identifier Restriction

Semantic analysis menjamin bahwa tidak ada manipulasi identitas lintas kategori. Ketika sebuah nama dievaluasi di dalam suatu ekspresi nilai (VarAccessNode) yaitu nama tersebut harus merujuk pada objek bertipe VARIABLE atau CONSTANT. Identitas yang dideklarasikan sebagai TYPE atau PROCEDURE tidak diizinkan muncul di dalam ekspresi nilai (seperti operasi matematika atau penugasan), kecuali identitas tersebut merujuk pada fungsi yang sedang dipanggil atau variabel return fungsi

7. Function Return Rule

Sebuah fungsi harus memberikan nilai kembali melalui penugasan ke identitas fungsi itu sendiri (Add := a + b). Tipe data dari nilai yang di-assign ke identitas fungsi harus assignment-compatible dengan return type fungsi tersebut.

Implementasi kelas SemanticVisitor dapat dilihat di repository dengan file SemanticVisitor.hpp dan SemanticVisitor.cpp

3. Pengujian

3.1. Kasus 1

```
program test1;
```

```
var
```

```
  x: Integer;
```

```
procedure SetLocal();
```

```
var
```

```
  x: Integer;
```

```
begin
```

```
  x := 10;
```

```
  if x > 5 then
```

```
    x := x + 1;
```

```
end;
```

```
begin
```

```
  x := 1;
```

```
  SetLocal();
```

```
End.
```

Global x ada di blok luar, lalu ada x lain di dalam procedure SetLocal. Karena ada deklarasi lokal dengan nama sama, semantic analyzer harus menganggap x di dalam prosedur sebagai simbol berbeda dari x global. Expected: pass.



=====

[0] Semantic Analysis completed successfully!

===== DISPLAY OPTIONS =====

1. Decorated AST only
 2. Symbol Table (tab) only
 3. Array Table (atab) only
 4. Block Table (btabs) only
 5. All semantic information
 6. Exit
- Choose: 5

COMPLETE SEMANTIC ANALYSIS REPORT

===== DECORATED AST =====

Program: test1
Scope Level: 0
Line: 0, Column: 0

Declarations: 2

- [0] VAR x (Line 0)
- [1] PROCEDURE SetLocal (Line 0)

===== AST Generated =====

ProgramNode(name: 'test1')

- VarDecl('x') → tab_index:39, type:integer, lev:0
 - type_def: SimpleTypeNode('Integer') → resolved_type:integer
- ProcedureDecl('SetLocal') → tab_index:40, lev:0, local_size:4
 - body: Block → block_index:1, lev:1
 - VarDecl('x') → tab_index:41, type:integer, lev:1
 - type_def: SimpleTypeNode('Integer') → resolved_type:integer
 - statements: CompoundStatement
 - Assign → type:void
 - target: VarAccess('x') → tab_index:41, type:integer, lev:1
 - value: IntLiteral(10) → type:integer
 - IfStatement
 - condition: BinOp('>') → type:boolean
 - left: VarAccess('x') → tab_index:41, type:integer, lev:1
 - right: IntLiteral(5) → type:integer
 - then: Assign → type:void
 - target: VarAccess('x') → tab_index:41, type:integer, lev:1
 - value: BinOp('+') → type:integer
 - left: VarAccess('x') → tab_index:41, type:integer, lev:1
 - right: IntLiteral(1) → type:integer
- main: CompoundStatement
 - Assign → type:void
 - target: VarAccess('x') → tab_index:39, type:integer, lev:0
 - value: IntLiteral(1) → type:integer
 - ProcedureCall('SetLocal') → tab_index:40, type:void

===== SYMBOL TABLE (TAB) =====

Index	Name	Type	Object	Level	Link	Ref	Nrm	Adr/Val
0	NULL	UNKNOWN	TYPE	0	0	0	0	0
1	INTEGER	INTEGER	TYPE	0	0	0	1	4
2	REAL	REAL	TYPE	0	1	0	1	8
3	CHAR	CHAR	TYPE	0	2	0	1	1
4	BOOLEAN	BOOLEAN	TYPE	0	3	0	1	1
5	STRING	STRING	TYPE	0	4	0	1	255
6	AND	VOID	RESERVE	0	5	0	0	0
7	NOT	VOID	RESERVE	0	6	0	0	0



0	NULL	UNKOWN	TYPE	0	0	0	0	0
1	INTEGER	INTEGER	TYPE	0	0	0	1	4
2	REAL	REAL	TYPE	0	1	0	1	8
3	CHAR	CHAR	TYPE	0	2	0	1	1
4	BOOLEAN	BOOLEAN	TYPE	0	3	0	1	1
5	STRING	STRING	TYPE	0	4	0	1	255
6	AND	VOID	RESERVE	0	5	0	0	0
7	NOT	VOID	RESERVE	0	6	0	0	0
8	MOD	VOID	RESERVE	0	7	0	0	0
9	DIV	VOID	RESERVE	0	8	0	0	0
10	IF	VOID	RESERVE	0	9	0	0	0
11	BEGIN	VOID	RESERVE	0	10	0	0	0
12	CASE	VOID	RESERVE	0	11	0	0	0
13	CONST	VOID	RESERVE	0	12	0	0	0
14	DOWNT0	VOID	RESERVE	0	13	0	0	0
15	DO	VOID	RESERVE	0	14	0	0	0
16	ELSE	VOID	RESERVE	0	15	0	0	0
17	END	VOID	RESERVE	0	16	0	0	0
18	FOR	VOID	RESERVE	0	17	0	0	0
19	OF	VOID	RESERVE	0	18	0	0	0
20	OR	VOID	RESERVE	0	19	0	0	0
21	PROGRAM	VOID	RESERVE	0	20	0	0	0
22	REPEAT	VOID	RESERVE	0	21	0	0	0
23	THEN	VOID	RESERVE	0	22	0	0	0
24	TO	VOID	RESERVE	0	23	0	0	0
25	TYPE	VOID	RESERVE	0	24	0	0	0
26	VAR	VOID	RESERVE	0	25	0	0	0
27	UNTIL	VOID	RESERVE	0	26	0	0	0
28	WHILE	VOID	RESERVE	0	27	0	0	0
29	ARRAY	VOID	RESERVE	0	28	0	0	0
30	RECORD	VOID	RESERVE	0	29	0	0	0
31	FUNCTION	VOID	RESERVE	0	30	0	0	0
32	PROCEDURE	VOID	RESERVE	0	31	0	0	0
33	writeln	VOID	PROCEDURE	0	32	0	0	0
34	write	VOID	PROCEDURE	0	33	0	0	0
35	read	VOID	PROCEDURE	0	34	0	0	0
36	readln	VOID	PROCEDURE	0	35	0	0	0
37	True	BOOLEAN	CONSTANT	0	36	0	0	0
38	False	BOOLEAN	CONSTANT	0	37	0	0	0
39	x	INTEGER	VARIABLE	0	38	0	0	0
40	SetLocal	UNKOWN	PROCEDURE	0	39	0	0	0
41	x	INTEGER	VARIABLE	1	40	0	0	0

===== ARRAY TABLE (ATAB) =====

Index	Index Type	Element Type	Elem Ref	Low	High	Elem Size	Total Size
0	UNKOWN	UNKOWN	0	0	0	0	0

===== BLOCK TABLE (BTAB) =====

Index	Last	Last Param	Param Size	Var Size
0	40	0	0	4
1	41	40	0	4

3.2. Kasus 2

```
program test2;
```

```
var
```

```
  i: Integer;
```

```
  r: Real;
```

```
  flag: Boolean;
```

```
  ch: Char;
```

```
  txt: String;
```

```
begin
```

```
  i := 3 + 4 * 2;
```

```
  r := 2.5 + 1.5;
```

```
  flag := True;
```

```
  ch := 'A';
```

```
  txt := 'Halo';
```

```
  if flag then
```

```
    i := i + 1
```

```
  else
```

```
    i := i - 1;
```

```
End.
```

Di assignment dan if flag then else. Cek apakah ekspresi aritmetika valid, literal sesuai tipe Integer, Real, Boolean, Char, String, dan kondisi if memang boolean. Expected: pass.



```
=====
[0] Semantic Analysis completed successfully!
```

```
===== DISPLAY OPTIONS =====
```

1. Decorated AST only
 2. Symbol Table (tab) only
 3. Array Table (atab) only
 4. Block Table (btabs) only
 5. All semantic information
 6. Exit
- Choose: 5

COMPLETE SEMANTIC ANALYSIS REPORT

```
===== DECORATED AST =====
```

```
Program: test2
Scope Level: 0
Line: 0, Column: 0
```

```
Declarations: 5
```

- [0] VAR i (Line 0)
- [1] VAR r (Line 0)
- [2] VAR flag (Line 0)
- [3] VAR ch (Line 0)
- [4] VAR txt (Line 0)

```
===== AST Generated =====
```

```
ProgramNode(name: 'test2')
```

```
├─ VarDecl('i') → tab_index:39, type:integer, lev:0
│   └─ type_def: SimpleTypeNode('Integer') → resolved_type:integer
├─ VarDecl('r') → tab_index:40, type:real, lev:0
│   └─ type_def: SimpleTypeNode('Real') → resolved_type:real
├─ VarDecl('flag') → tab_index:41, type:boolean, lev:0
│   └─ type_def: SimpleTypeNode('Boolean') → resolved_type:boolean
├─ VarDecl('ch') → tab_index:42, type:char, lev:0
│   └─ type_def: SimpleTypeNode('Char') → resolved_type:char
├─ VarDecl('txt') → tab_index:43, type:string, lev:0
│   └─ type_def: SimpleTypeNode('String') → resolved_type:string
└─ main: CompoundStatement
    ├─ Assign → type:void
    │   ├── target: VarAccess('i') → tab_index:39, type:integer, lev:0
    │   └─ value: BinOp('+') → type:integer
    │       ├── left: IntLiteral(3) → type:integer
    │       └─ right: BinOp('*') → type:integer
    │           ├── left: IntLiteral(4) → type:integer
    │           └─ right: IntLiteral(2) → type:integer
    ├─ Assign → type:void
    │   ├── target: VarAccess('r') → tab_index:40, type:real, lev:0
    │   └─ value: BinOp('+') → type:real
    │       ├── left: RealLiteral(2.500000) → type:real
    │       └─ right: RealLiteral(1.500000) → type:real
    ├─ Assign → type:void
    │   ├── target: VarAccess('flag') → tab_index:41, type:boolean, lev:0
    │   └─ value: VarAccess('True') → tab_index:37, type:boolean, CONSTANT, lev:0
    ├─ Assign → type:void
    │   ├── target: VarAccess('ch') → tab_index:42, type:char, lev:0
    │   └─ value: CharLiteral('A') → type:char
    ├─ Assign → type:void
    │   ├── target: VarAccess('txt') → tab_index:43, type:string, lev:0
    │   └─ value: StringLiteral('Halo') → type:string
    └─ IfStatement
```




0	NULL	UNKOWN	TYPE	0	0	0	0	0
1	INTEGER	INTEGER	TYPE	0	0	0	1	4
2	REAL	REAL	TYPE	0	1	0	1	8
3	CHAR	CHAR	TYPE	0	2	0	1	1
4	BOOLEAN	BOOLEAN	TYPE	0	3	0	1	1
5	STRING	STRING	TYPE	0	4	0	1	255
6	AND	VOID	RESERVE	0	5	0	0	0
7	NOT	VOID	RESERVE	0	6	0	0	0
8	MOD	VOID	RESERVE	0	7	0	0	0
9	DIV	VOID	RESERVE	0	8	0	0	0
10	IF	VOID	RESERVE	0	9	0	0	0
11	BEGIN	VOID	RESERVE	0	10	0	0	0
12	CASE	VOID	RESERVE	0	11	0	0	0
13	CONST	VOID	RESERVE	0	12	0	0	0
14	DOWNT0	VOID	RESERVE	0	13	0	0	0
15	DO	VOID	RESERVE	0	14	0	0	0
16	ELSE	VOID	RESERVE	0	15	0	0	0
17	END	VOID	RESERVE	0	16	0	0	0
18	FOR	VOID	RESERVE	0	17	0	0	0
19	OF	VOID	RESERVE	0	18	0	0	0
20	OR	VOID	RESERVE	0	19	0	0	0
21	PROGRAM	VOID	RESERVE	0	20	0	0	0
22	REPEAT	VOID	RESERVE	0	21	0	0	0
23	THEN	VOID	RESERVE	0	22	0	0	0
24	TO	VOID	RESERVE	0	23	0	0	0
25	TYPE	VOID	RESERVE	0	24	0	0	0
26	VAR	VOID	RESERVE	0	25	0	0	0
27	UNTIL	VOID	RESERVE	0	26	0	0	0
28	WHILE	VOID	RESERVE	0	27	0	0	0
29	ARRAY	VOID	RESERVE	0	28	0	0	0
30	RECORD	VOID	RESERVE	0	29	0	0	0
31	FUNCTION	VOID	RESERVE	0	30	0	0	0
32	PROCEDURE	VOID	RESERVE	0	31	0	0	0
33	writeln	VOID	PROCEDURE	0	32	0	0	0
34	write	VOID	PROCEDURE	0	33	0	0	0
35	read	VOID	PROCEDURE	0	34	0	0	0
36	readln	VOID	PROCEDURE	0	35	0	0	0
37	True	BOOLEAN	CONSTANT	0	36	0	0	0
38	False	BOOLEAN	CONSTANT	0	37	0	0	0
39	i	INTEGER	VARIABLE	0	38	0	0	0
40	r	REAL	VARIABLE	0	39	0	0	0
41	flag	BOOLEAN	VARIABLE	0	40	0	0	0
42	ch	CHAR	VARIABLE	0	41	0	0	0
43	txt	STRING	VARIABLE	0	42	0	0	0

=====

===== ARRAY TABLE (ATAB) =====

Index	Index Type	Element Type	Elem Ref	Low	High	Elem Size	Total Size
0	UNKOWN	UNKOWN	0	0	0	0	0

=====

===== BLOCK TABLE (BTAB) =====

Index	Last	Last Param	Param Size	Var Size
0	43	0	0	270

=====

3.3. Kasus 3

```
program test3;

type
  IntArray == array [1..5] of Integer;

var
  arr: IntArray;
  i, total: Integer;

begin
  arr[1] := 10;
  arr[2] := 20;
  arr[3] := 30;
  total := arr[1] + arr[2] + arr[3];

  for i := 1 to 3 do
    begin
      total := total + arr[i];
    end;
  end;
End.
```

menguji deklarasi dan akses larik. IntArray == array [1..5] of Integer adalah deklarasi array, lalu akses arr[1], arr[2], arr[3], dan arr[i] menguji indexing. Karena variabelnya array dan indexnya dipakai benar, ini harus lolos. Expected: pass.



```
=====
[0] Semantic Analysis completed successfully!

===== DISPLAY OPTIONS =====
1. Decorated AST only
2. Symbol Table (tab) only
3. Array Table (atab) only
4. Block Table (btabs) only
5. All semantic information
6. Exit
Choose: 5
      COMPLETE SEMANTIC ANALYSIS REPORT

===== DECORATED AST =====
Program: test3
Scope Level: 0
Line: 0, Column: 0

Declarations: 4
[0] TYPE IntArray (Line 0)
[1] VAR arr (Line 0)
[2] VAR i (Line 0)
[3] VAR total (Line 0)

===== AST Generated =====
ProgramNode(name: 'test3')
├─ TypeDecl('IntArray') → tab_index:39, type:array, lev:0
│   └─ type_def: ArrayTypeNode [atab_ref:1] → resolved_type:array
│       ├── index_type: RangeTypeNode [base_type:integer] → resolved_type:subrange
│       │   ├── low: IntLiteral(1) → type:integer
│       │   └─ high: IntLiteral(5) → type:integer
│       └─ element_type: SimpleTypeNode('Integer') → resolved_type:integer
├─ VarDecl('arr') → tab_index:40, type:array, lev:0
│   └─ type_def: SimpleTypeNode('IntArray') → resolved_type:array
├─ VarDecl('i') → tab_index:41, type:integer, lev:0
│   └─ type_def: SimpleTypeNode('Integer') → resolved_type:integer
├─ VarDecl('total') → tab_index:42, type:integer, lev:0
│   └─ type_def: SimpleTypeNode('Integer') → resolved_type:integer
└─ main: CompoundStatement
    ├── Assign → type:void
    │   ├── target: ArrayAccess → type:integer
    │   │   ├── target: VarAccess('arr') → tab_index:40, type:array, lev:0
    │   │   └─ index: IntLiteral(1) → type:integer
    │   └─ value: IntLiteral(10) → type:integer
    ├── Assign → type:void
    │   ├── target: ArrayAccess → type:integer
    │   │   ├── target: VarAccess('arr') → tab_index:40, type:array, lev:0
    │   │   └─ index: IntLiteral(2) → type:integer
    │   └─ value: IntLiteral(20) → type:integer
    ├── Assign → type:void
    │   ├── target: ArrayAccess → type:integer
    │   │   ├── target: VarAccess('arr') → tab_index:40, type:array, lev:0
    │   │   └─ index: IntLiteral(3) → type:integer
    │   └─ value: IntLiteral(30) → type:integer
    └─ Assign → type:void
        ├── target: VarAccess('total') → tab_index:42, type:integer, lev:0
        └─ value: BinOp('+') → type:integer
            ├── left: BinOp('+') → type:integer
            │   ├── left: ArrayAccess → type:integer
            │   │   ├── target: VarAccess('arr') → tab_index:40, type:array, lev:0
            │   │   └─ index: IntLiteral(1) → type:integer
            │   └─ right: ArrayAccess → type:integer
```



```
└─ Assign → type:void
  └─ target: ArrayAccess → type:integer
    └─ target: VarAccess('arr') → tab_index:40, type:array, lev:0
      └─ index: IntLiteral(2) → type:integer
    value: IntLiteral(20) → type:integer
└─ Assign → type:void
  └─ target: ArrayAccess → type:integer
    └─ target: VarAccess('arr') → tab_index:40, type:array, lev:0
      └─ index: IntLiteral(3) → type:integer
    value: IntLiteral(30) → type:integer
└─ Assign → type:void
  └─ target: VarAccess('total') → tab_index:42, type:integer, lev:0
    value: BinOp('+') → type:integer
      └─ left: BinOp('+') → type:integer
        └─ left: ArrayAccess → type:integer
          └─ target: VarAccess('arr') → tab_index:40, type:array, lev:0
            └─ index: IntLiteral(1) → type:integer
        right: ArrayAccess → type:integer
          └─ target: VarAccess('arr') → tab_index:40, type:array, lev:0
            └─ index: IntLiteral(2) → type:integer
      right: ArrayAccess → type:integer
        └─ target: VarAccess('arr') → tab_index:40, type:array, lev:0
          └─ index: IntLiteral(3) → type:integer
└─ ForLoop('i' to)
  └─ start: IntLiteral(1) → type:integer
  └─ end: IntLiteral(3) → type:integer
  body: CompoundStatement
    └─ Assign → type:void
      └─ target: VarAccess('total') → tab_index:42, type:integer, lev:0
        value: BinOp('+') → type:integer
          └─ left: VarAccess('total') → tab_index:42, type:integer, lev:0
            └─ right: ArrayAccess → type:integer
              └─ target: VarAccess('arr') → tab_index:40, type:array, lev:0
                └─ index: VarAccess('i') → tab_index:41, type:integer, lev:0
```

===== SYMBOL TABLE (TAB) =====

Index	Name	Type	Object	Level	Link	Ref	Nrm	Adr/Val
0	NULL	UNKNOWN	TYPE	0	0	0	0	0
1	INTEGER	INTEGER	TYPE	0	0	0	1	4
2	REAL	REAL	TYPE	0	1	0	1	8
3	CHAR	CHAR	TYPE	0	2	0	1	1
4	BOOLEAN	BOOLEAN	TYPE	0	3	0	1	1
5	STRING	STRING	TYPE	0	4	0	1	255
6	AND	VOID	RESERVE	0	5	0	0	0
7	NOT	VOID	RESERVE	0	6	0	0	0
8	MOD	VOID	RESERVE	0	7	0	0	0
9	DIV	VOID	RESERVE	0	8	0	0	0
10	IF	VOID	RESERVE	0	9	0	0	0
11	BEGIN	VOID	RESERVE	0	10	0	0	0
12	CASE	VOID	RESERVE	0	11	0	0	0
13	CONST	VOID	RESERVE	0	12	0	0	0
14	DOWNT0	VOID	RESERVE	0	13	0	0	0
15	DO	VOID	RESERVE	0	14	0	0	0
16	ELSE	VOID	RESERVE	0	15	0	0	0
17	END	VOID	RESERVE	0	16	0	0	0
18	FOR	VOID	RESERVE	0	17	0	0	0
19	OF	VOID	RESERVE	0	18	0	0	0
20	OR	VOID	RESERVE	0	19	0	0	0



0	NULL	UNKOWN	TYPE	0	0	0	0	0
1	INTEGER	INTEGER	TYPE	0	0	0	1	4
2	REAL	REAL	TYPE	0	1	0	1	8
3	CHAR	CHAR	TYPE	0	2	0	1	1
4	BOOLEAN	BOOLEAN	TYPE	0	3	0	1	1
5	STRING	STRING	TYPE	0	4	0	1	255
6	AND	VOID	RESERVE	0	5	0	0	0
7	NOT	VOID	RESERVE	0	6	0	0	0
8	MOD	VOID	RESERVE	0	7	0	0	0
9	DIV	VOID	RESERVE	0	8	0	0	0
10	IF	VOID	RESERVE	0	9	0	0	0
11	BEGIN	VOID	RESERVE	0	10	0	0	0
12	CASE	VOID	RESERVE	0	11	0	0	0
13	CONST	VOID	RESERVE	0	12	0	0	0
14	DOWNT0	VOID	RESERVE	0	13	0	0	0
15	DO	VOID	RESERVE	0	14	0	0	0
16	ELSE	VOID	RESERVE	0	15	0	0	0
17	END	VOID	RESERVE	0	16	0	0	0
18	FOR	VOID	RESERVE	0	17	0	0	0
19	OF	VOID	RESERVE	0	18	0	0	0
20	OR	VOID	RESERVE	0	19	0	0	0
21	PROGRAM	VOID	RESERVE	0	20	0	0	0
22	REPEAT	VOID	RESERVE	0	21	0	0	0
23	THEN	VOID	RESERVE	0	22	0	0	0
24	TO	VOID	RESERVE	0	23	0	0	0
25	TYPE	VOID	RESERVE	0	24	0	0	0
26	VAR	VOID	RESERVE	0	25	0	0	0
27	UNTIL	VOID	RESERVE	0	26	0	0	0
28	WHILE	VOID	RESERVE	0	27	0	0	0
29	ARRAY	VOID	RESERVE	0	28	0	0	0
30	RECORD	VOID	RESERVE	0	29	0	0	0
31	FUNCTION	VOID	RESERVE	0	30	0	0	0
32	PROCEDURE	VOID	RESERVE	0	31	0	0	0
33	writeln	VOID	PROCEDURE	0	32	0	0	0
34	write	VOID	PROCEDURE	0	33	0	0	0
35	read	VOID	PROCEDURE	0	34	0	0	0
36	readln	VOID	PROCEDURE	0	35	0	0	0
37	True	BOOLEAN	CONSTANT	0	36	0	0	0
38	False	BOOLEAN	CONSTANT	0	37	0	0	0
39	IntArray	ARRAY	TYPE	0	38	1	0	0
40	arr	ARRAY	VARIABLE	0	39	1	0	0
41	i	INTEGER	VARIABLE	0	40	0	0	0
42	total	INTEGER	VARIABLE	0	41	0	0	0

===== ARRAY TABLE (ATAB) =====

Index	Index Type	Element Type	Elem Ref	Low	High	Elem Size	Total Size
0	UNKOWN	UNKOWN	0	0	0	0	0
1	INTEGER	INTEGER	0	1	5	4	20

===== BLOCK TABLE (BTAB) =====

Index	Last	Last Param	Param Size	Var Size
0	42	0	0	28

3.4. Kasus 4

```
program test4;
```

```
type
```

```
  Person == record
```

```
    nama: String;
```

```
    umur: Integer;
```

```
  end;
```

```
var
```

```
  p: Person;
```

```
begin
```

```
  p.nama := 'Budi';
```

```
  p.umur := 18;
```

```
End.
```

Menguji deklarasi record + akses atribut. Person == record mendefinisikan field nama dan umur, lalu p.nama dan p.umur di baris 13-14 memastikan atribut record di-resolve ke field yang memang ada. Expected: pass.



=====
[0] Semantic Analysis completed successfully!

===== DISPLAY OPTIONS =====

1. Decorated AST only
2. Symbol Table (tab) only
3. Array Table (atab) only
4. Block Table (btabs) only
5. All semantic information
6. Exit

Choose: 5

COMPLETE SEMANTIC ANALYSIS REPORT

===== DECORATED AST =====

Program: test4

Scope Level: 0

Line: 0, Column: 0

Declarations: 2

[0] TYPE Person (Line 0)

[1] VAR p (Line 0)

===== AST Generated =====

ProgramNode(name: 'test4')

```
├─ TypeDecl('Person') → tab_index:41, type:record, lev:0
│   └─ type_def: RecordTypeNode [btabs_ref:1] → resolved_type:record
│       ├── field: VarDecl('nama') → tab_index:39, type:string, lev:0
│       │   └─ type_def: SimpleTypeNode('String') → resolved_type:string
│       └─ field: VarDecl('umur') → tab_index:40, type:integer, lev:0
│           └─ type_def: SimpleTypeNode('Integer') → resolved_type:integer
├─ VarDecl('p') → tab_index:42, type:record, lev:0
│   └─ type_def: SimpleTypeNode('Person') → resolved_type:record
└─ main: CompoundStatement
    ├── Assign → type:void
    │   ├── target: FieldAccess('.nama') → tab_index:39, type:string
    │   │   └─ target: VarAccess('p') → tab_index:42, type:record, lev:0
    │   └─ value: StringLiteral('Budi') → type:string
    └─ Assign → type:void
        ├── target: FieldAccess('.umur') → tab_index:40, type:integer
        │   └─ target: VarAccess('p') → tab_index:42, type:record, lev:0
        └─ value: IntLiteral(18) → type:integer
```

=====

===== SYMBOL TABLE (TAB) =====

Index	Name	Type	Object	Level	Link	Ref	Nrm	Adr/Val
0	NULL	UNKNOWN	TYPE	0	0	0	0	0
1	INTEGER	INTEGER	TYPE	0	0	0	1	4
2	REAL	REAL	TYPE	0	1	0	1	8
3	CHAR	CHAR	TYPE	0	2	0	1	1
4	BOOLEAN	BOOLEAN	TYPE	0	3	0	1	1
5	STRING	STRING	TYPE	0	4	0	1	255
6	AND	VOID	RESERVE	0	5	0	0	0
7	NOT	VOID	RESERVE	0	6	0	0	0
8	MOD	VOID	RESERVE	0	7	0	0	0
9	DIV	VOID	RESERVE	0	8	0	0	0
10	IF	VOID	RESERVE	0	9	0	0	0
11	BEGIN	VOID	RESERVE	0	10	0	0	0
12	CASE	VOID	RESERVE	0	11	0	0	0
13	CONST	VOID	RESERVE	0	12	0	0	0
14	DOWNT0	VOID	RESERVE	0	13	0	0	0



0	NULL	UNKOWN	TYPE	0	0	0	0	0
1	INTEGER	INTEGER	TYPE	0	0	0	1	4
2	REAL	REAL	TYPE	0	1	0	1	8
3	CHAR	CHAR	TYPE	0	2	0	1	1
4	BOOLEAN	BOOLEAN	TYPE	0	3	0	1	1
5	STRING	STRING	TYPE	0	4	0	1	255
6	AND	VOID	RESERVE	0	5	0	0	0
7	NOT	VOID	RESERVE	0	6	0	0	0
8	MOD	VOID	RESERVE	0	7	0	0	0
9	DIV	VOID	RESERVE	0	8	0	0	0
10	IF	VOID	RESERVE	0	9	0	0	0
11	BEGIN	VOID	RESERVE	0	10	0	0	0
12	CASE	VOID	RESERVE	0	11	0	0	0
13	CONST	VOID	RESERVE	0	12	0	0	0
14	DOWNT0	VOID	RESERVE	0	13	0	0	0
15	DO	VOID	RESERVE	0	14	0	0	0
16	ELSE	VOID	RESERVE	0	15	0	0	0
17	END	VOID	RESERVE	0	16	0	0	0
18	FOR	VOID	RESERVE	0	17	0	0	0
19	OF	VOID	RESERVE	0	18	0	0	0
20	OR	VOID	RESERVE	0	19	0	0	0
21	PROGRAM	VOID	RESERVE	0	20	0	0	0
22	REPEAT	VOID	RESERVE	0	21	0	0	0
23	THEN	VOID	RESERVE	0	22	0	0	0
24	TO	VOID	RESERVE	0	23	0	0	0
25	TYPE	VOID	RESERVE	0	24	0	0	0
26	VAR	VOID	RESERVE	0	25	0	0	0
27	UNTIL	VOID	RESERVE	0	26	0	0	0
28	WHILE	VOID	RESERVE	0	27	0	0	0
29	ARRAY	VOID	RESERVE	0	28	0	0	0
30	RECORD	VOID	RESERVE	0	29	0	0	0
31	FUNCTION	VOID	RESERVE	0	30	0	0	0
32	PROCEDURE	VOID	RESERVE	0	31	0	0	0
33	writeln	VOID	PROCEDURE	0	32	0	0	0
34	write	VOID	PROCEDURE	0	33	0	0	0
35	read	VOID	PROCEDURE	0	34	0	0	0
36	readln	VOID	PROCEDURE	0	35	0	0	0
37	True	BOOLEAN	CONSTANT	0	36	0	0	0
38	False	BOOLEAN	CONSTANT	0	37	0	0	0
39	nama	STRING	VARIABLE	0	0	0	0	0
40	umur	INTEGER	VARIABLE	0	39	0	0	0
41	Person	RECORD	TYPE	0	38	1	0	0
42	p	RECORD	VARIABLE	0	41	1	0	0

===== ARRAY TABLE (ATAB) =====

Index	Index Type	Element Type	Elem Ref	Low	High	Elem Size	Total Size
0	UNKOWN	UNKOWN	0	0	0	0	0

===== BLOCK TABLE (BTAB) =====

Index	Last	Last Param	Param Size	Var Size
0	42	0	0	260
1	40	-1	0	260

3.5. Kasus 5

```
program test5;

var
  hasil: Integer;

function Add(a: Integer; b: Integer): Integer;
begin
  Add := a + b;
end;

begin
  hasil := Add(2, 3);
end.
```

Menguji deklarasi fungsi + pemanggilan. Bagian penting ada di function Add(a: Integer; b: Integer): Integer; dan assignment Add := a + b, lalu pemanggilan Add(2, 3). Mengecek parameter function, return type, dan call yang argumennya cocok. Expected: pass.



```
=====
[0] Semantic Analysis completed successfully!
```

```
===== DISPLAY OPTIONS =====
```

1. Decorated AST only
2. Symbol Table (tab) only
3. Array Table (atab) only
4. Block Table (btav) only
5. All semantic information
6. Exit

```
Choose: 5
```

COMPLETE SEMANTIC ANALYSIS REPORT

```
===== DECORATED AST =====
```

```
Program: test5
```

```
Scope Level: 0
```

```
Line: 0, Column: 0
```

```
Declarations: 2
```

```
[0] VAR hasil (Line 0)
```

```
[1] FUNCTION Add (Line 0)
```

```
===== AST Generated =====
```

```
ProgramNode(name: 'test5')
```

```
├─ VarDecl('hasil') → tab_index:39, type:integer, lev:0
│   └─ type_def: SimpleTypeNode('Integer') → resolved_type:integer
├─ FunctionDecl('Add') → tab_index:40, return_type:integer, lev:0
│   ├── param: VarDecl('a') → tab_index:42, type:integer, lev:1
│   │   └─ type_def: SimpleTypeNode('Integer') → resolved_type:integer
│   ├── param: VarDecl('b') → tab_index:43, type:integer, lev:1
│   │   └─ type_def: SimpleTypeNode('Integer') → resolved_type:integer
│   ├── return: SimpleTypeNode('Integer') → resolved_type:integer
│   └─ body: Block → block_index:1, lev:1
│       └─ statements: CompoundStatement
│           └─ Assign → type:void
│               ├── target: VarAccess('Add') → tab_index:41, type:integer, lev:1
│               └─ value: BinOp('+') → type:integer
│                   ├── left: VarAccess('a') → tab_index:42, type:integer, lev:1
│                   └─ right: VarAccess('b') → tab_index:43, type:integer, lev:1
└─ main: CompoundStatement
    └─ Assign → type:void
        ├── target: VarAccess('hasil') → tab_index:39, type:integer, lev:0
        └─ value: FunctionCall('Add') → tab_index:40, type:integer
            ├── arg: IntLiteral(2) → type:integer
            └─ arg: IntLiteral(3) → type:integer
```

```
===== SYMBOL TABLE (TAB) =====
```

Index	Name	Type	Object	Level	Link	Ref	Nrm	Adr/Val
0	NULL	UNKNOWN	TYPE	0	0	0	0	0
1	INTEGER	INTEGER	TYPE	0	0	0	1	4
2	REAL	REAL	TYPE	0	1	0	1	8
3	CHAR	CHAR	TYPE	0	2	0	1	1
4	BOOLEAN	BOOLEAN	TYPE	0	3	0	1	1
5	STRING	STRING	TYPE	0	4	0	1	255
6	AND	VOID	RESERVE	0	5	0	0	0
7	NOT	VOID	RESERVE	0	6	0	0	0
8	MOD	VOID	RESERVE	0	7	0	0	0
9	DIV	VOID	RESERVE	0	8	0	0	0
10	IF	VOID	RESERVE	0	9	0	0	0



1	INTEGER	INTEGER	TYPE	0	0	0	1	4
2	REAL	REAL	TYPE	0	1	0	1	8
3	CHAR	CHAR	TYPE	0	2	0	1	1
4	BOOLEAN	BOOLEAN	TYPE	0	3	0	1	1
5	STRING	STRING	TYPE	0	4	0	1	255
6	AND	VOID	RESERVE	0	5	0	0	0
7	NOT	VOID	RESERVE	0	6	0	0	0
8	MOD	VOID	RESERVE	0	7	0	0	0
9	DIV	VOID	RESERVE	0	8	0	0	0
10	IF	VOID	RESERVE	0	9	0	0	0
11	BEGIN	VOID	RESERVE	0	10	0	0	0
12	CASE	VOID	RESERVE	0	11	0	0	0
13	CONST	VOID	RESERVE	0	12	0	0	0
14	DOWNT0	VOID	RESERVE	0	13	0	0	0
15	DO	VOID	RESERVE	0	14	0	0	0
16	ELSE	VOID	RESERVE	0	15	0	0	0
17	END	VOID	RESERVE	0	16	0	0	0
18	FOR	VOID	RESERVE	0	17	0	0	0
19	OF	VOID	RESERVE	0	18	0	0	0
20	OR	VOID	RESERVE	0	19	0	0	0
21	PROGRAM	VOID	RESERVE	0	20	0	0	0
22	REPEAT	VOID	RESERVE	0	21	0	0	0
23	THEN	VOID	RESERVE	0	22	0	0	0
24	TO	VOID	RESERVE	0	23	0	0	0
25	TYPE	VOID	RESERVE	0	24	0	0	0
26	VAR	VOID	RESERVE	0	25	0	0	0
27	UNTIL	VOID	RESERVE	0	26	0	0	0
28	WHILE	VOID	RESERVE	0	27	0	0	0
29	ARRAY	VOID	RESERVE	0	28	0	0	0
30	RECORD	VOID	RESERVE	0	29	0	0	0
31	FUNCTION	VOID	RESERVE	0	30	0	0	0
32	PROCEDURE	VOID	RESERVE	0	31	0	0	0
33	writeln	VOID	PROCEDURE	0	32	0	0	0
34	write	VOID	PROCEDURE	0	33	0	0	0
35	read	VOID	PROCEDURE	0	34	0	0	0
36	readln	VOID	PROCEDURE	0	35	0	0	0
37	True	BOOLEAN	CONSTANT	0	36	0	0	0
38	False	BOOLEAN	CONSTANT	0	37	0	0	0
39	hasil	INTEGER	VARIABLE	0	38	0	0	0
40	Add	INTEGER	FUNCTION	0	39	0	0	0
41	Add	INTEGER	VARIABLE	1	40	0	0	0
42	a	INTEGER	VARIABLE	1	41	0	0	0
43	b	INTEGER	VARIABLE	1	42	0	0	0

===== ARRAY TABLE (ATAB) =====

Index	Index Type	Element Type	Elem Ref	Low	High	Elem Size	Total Size
0	UNKNOWN	UNKNOWN	0	0	0	0	0

===== BLOCK TABLE (BTAB) =====

Index	Last	Last Param	Param Size	Var Size
0	40	0	0	4
1	43	43	8	0

3.6. Kasus 6

```
program test6;
```

```
type
```

```
  Person == record
```

```
    nama: String;
```

```
    umur: Integer;
```

```
  end;
```

```
var
```

```
  p: Person;
```

```
begin
```

```
  p.nama := 'Budi';
```

```
  p.alamat := 'Bandung';
```

```
end.
```

Menguji akses atribut record yang tidak ada. Person hanya punya nama dan umur, tapi ada yang menggunakan p.alamat. Semantic analyzer harus menolak karena field alamat tidak pernah dideklarasikan di record itu. Expected: fail.

```
=====
```

```
[!] Semantic Analysis completed with errors:
```

```
Semantic Errors:
```

```
  Line 14, Column 5: [Semantic Error] at Line 14, Col 5: Undefined record field: alamat
```

```
  Line 14, Column 5: [Semantic Error] at Line 14, Col 5: Type mismatch in assignment
```

```
[!] Semantic analysis completed with errors.
```

```
===== DISPLAY OPTIONS =====
```

```
1. Decorated AST only
```

```
2. Symbol Table (tab) only
```

```
3. Array Table (atab) only
```

```
4. Block Table (btabs) only
```

```
5. All semantic information
```

```
6. Exit
```

```
Choose: █
```

3.7. Kasus 7

```
program test7;
```

```
var
```

```
x: Integer;
```

```
begin
```

```
  x := 10;
```

```
  x[1] := 5;
```

```
End.
```

Menguji indexing pada variabel non-larik. x dideklarasikan sebagai Integer, lalu dipakai x[1]. Karena x bukan array, indexing harus dianggap error semantic. Expected: fail.

```
=====
[!] Semantic Analysis completed with errors:
Semantic Errors:
  Line 9, Column 5: [Semantic Error] at Line 9, Col 5: Type mismatch in assignment
[!] Semantic analysis completed with errors.

===== DISPLAY OPTIONS =====
1. Decorated AST only
2. Symbol Table (tab) only
3. Array Table (atab) only
4. Block Table (btab) only
5. All semantic information
6. Exit
Choose: █
```

3.8. Kasus 8

```
program test9;
```

```
var
```

```
  angka: Integer;
```

```
begin
```

```
  angka := 'teks';
```

```
End.
```

Menguji assignment ke tipe tidak cocok. angka bertipe Integer, tapi ada angka := 'teks'. Ini harus ditolak karena assignment dari String ke Integer tidak kompatibel. Expected: fail.

```

=====

[!] Semantic Analysis completed with errors:
Semantic Errors:
  Line 7, Column 3: [Semantic Error] at Line 7, Col 3: Type mismatch in assignment
[!] Semantic analysis completed with errors.

===== DISPLAY OPTIONS =====
1. Decorated AST only
2. Symbol Table (tab) only
3. Array Table (atab) only
4. Block Table (btabs) only
5. All semantic information
6. Exit
Choose: █

```

3.9. Kasus 9

```
program test10;
```

```
begin
```

```
  x := 5;
```

```
end.
```

menguji pemakaian variabel yang belum dideklarasikan. Semantic analyzer harus mendeteksi identifier yang belum masuk symbol table. Expected: fail.

```

=====

[!] Semantic Analysis completed with errors:
Semantic Errors:
  Line 4, Column 3: [Semantic Error] at Line 4, Col 3: Undefined identifier: x
  Line 4, Column 3: [Semantic Error] at Line 4, Col 3: Type mismatch in assignment
[!] Semantic analysis completed with errors.

===== DISPLAY OPTIONS =====
1. Decorated AST only
2. Symbol Table (tab) only
3. Array Table (atab) only
4. Block Table (btabs) only
5. All semantic information
6. Exit
Choose: █

```

4. Kesimpulan dan Saran

4.1. Kesimpulan

Pada Milestone 3 ini, kami diimplementasikan Semantic Analysis untuk compiler bahasa Arion. Beberapa hal yang dapat disimpulkan dari proses perancangan dan implementasi yang telah dilakukan adalah sebagai berikut.

Pertama, semantic analyzer berhasil dibangun dengan arsitektur yang modular, terdiri dari komponen AST Builder, Symbol Table, AST Visitor (SemanticVisitor), dan ASTPrinter yang terintegrasi dalam SemanticAnalyzer.

Kedua, implementasi Visitor Pattern memisahkan logika semantic analysis dari struktur data AST. Dengan double dispatch melalui metode accept(), setiap jenis node AST dapat diproses secara spesifik tanpa mengubah struktur kelas node itu sendiri, sehingga sistem bersifat extensibl.

Ketiga, Symbol Table yang terdiri dari tiga tabel (tab, atab, btab) dapat merepresentasikan seluruh jenis identifier dalam bahasa Arion, termasuk variabel, konstanta, fungsi, prosedur, array, dan record, beserta informasi scope dan tipe datanya.

Keempat, berdasarkan hasil pengujian terhadap sembilan kasus uji, semantic analyzer mampu mendeteksi berbagai jenis kesalahan semantik, kasus-kasus yang seharusnya lolos atau pass juga berhasil diproses tanpa error.

4.2. Saran

- Billie Bhaskara Wibawa - 13524024



- Tubes is the friends we make along the way

- Raysha Erviandika Putra - 13524050



- If arion code has more than 1 line of text, disables it

- Muhammad Naufal Romi Annafi - 13524058



[peak](#)

“one must imagine if student happy”

- Dzaki Ahmad Al Hussainy - 13524084

Aku jam 1 pagi membaca buku Engineering Compiler, “Aduhh apa sih aturan semantik aneh ini..” tiba-tiba Citlali disampingku mengatakan “Ih udah lahh kamu tidur aja, Bahasa Arion ini aneh banget, aku udah ngantuk!!” sambil marah. “Ehh,ehh sabar sayang jangan begitu ini untuk tugas besar 4 sks aku lohh, kalau jelek IPK aku bisa turun banget aku harus lock-in” aku jawab dengan tegas. “Hmmp, yaudalah aku tidur duluan aja!” Citlali menjawab aku dengan nada agak marah. Terus kemudian aku mengatakan “Ehh jangan gitu kamu temenin aku dikit lagi deh”

aku mau nyelesain bikin Symbol table habis itu kita tidur gimana?". Kemudian Citlali dengan agak ngantuk mengatakan "yaudah deh dikit lagi yaa!" Kemudian Citlali sambil bersandar ke kiriku dan kemudian tertidur. Tetapi sebenarnya tubes ini masih banyak yang harus dikerjakan.

5. Lampiran

5.1. Repository

Github: [JEE-Tubes-IF2224-2026](#)

5.2. Tabel Pembagian Tugas

Nama	NIM	Workload	Persentase
Billie Bhaskara Wibawa	13524024	Laporan, Koding	25%
Raysha Erviandika Putra	13524050	Laporan, Koding	25%
Muhammad Naufal Romi Annafi	13524058	Laporan, Koding	25%
Dzaki Ahmad Al Hussainy	13524084	Laporan, Koding	25%

6. Referensi

"Semantic Analysis (ULiège)"

<https://people.montefiore.uliege.be/geurts/Cours/compil/2015/04-semantic-2015-2016.pdf>